

GamutX

Python Libraries

PYTHON SCRIPTING

Introduction to Python Libraries

- What Are Python Libraries?
- Types of Python Libraries
- When Do We Need to Find a Library?
- Finding Python Libraries Online
- What is PyPI?

What Are Python Libraries?

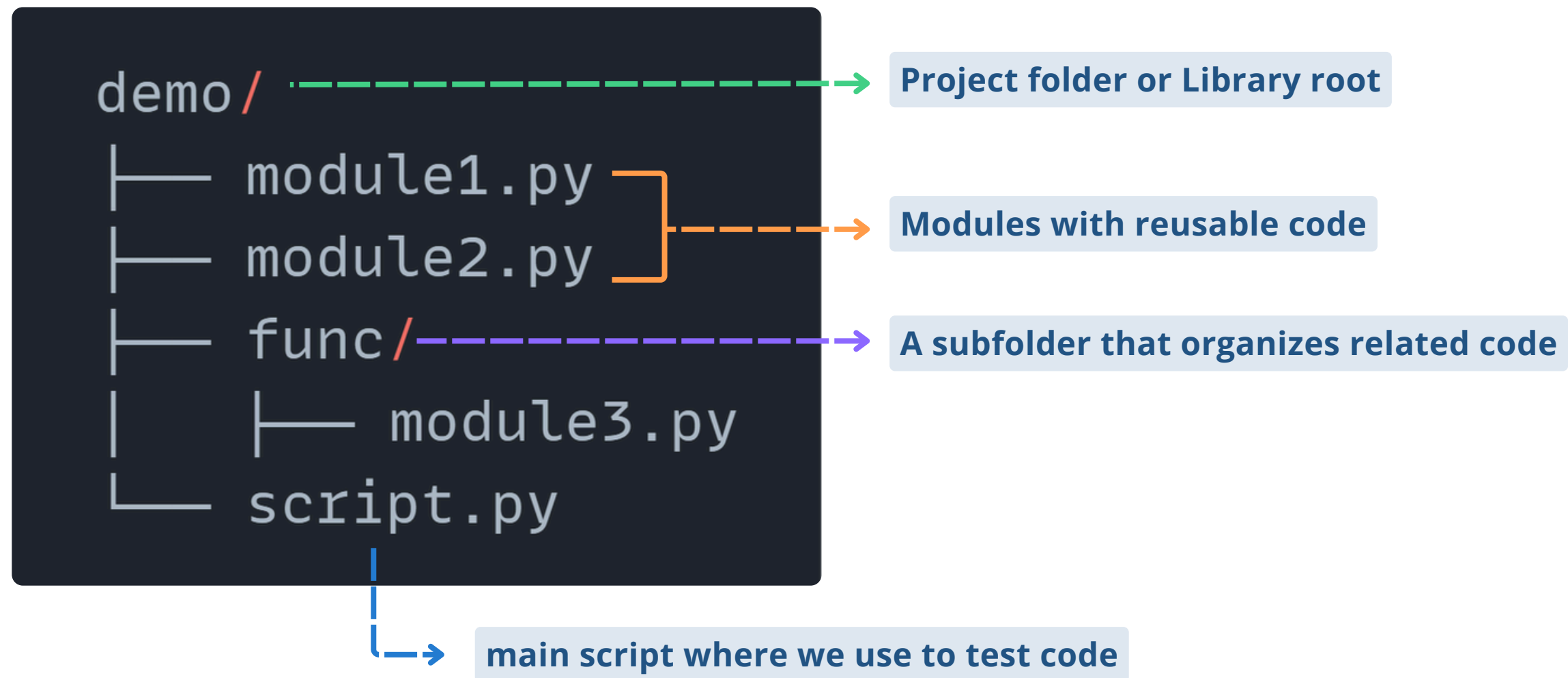
- A **Python library** is a collection of code written by someone else to help you perform specific tasks.
- Instead of writing everything from scratch, we **reuse** code by importing it into our own project.
- These libraries often solve common problems
 - like **math operations**, **file handling**, or **building user interfaces**.

Libraries usually contain:

- **Modules:**
 - A single Python file (e.g., module1.py)
 - Contains functions, classes, and variables — all reusable
- **A Library:**
 - A group of modules
 - Organized in folders (sometimes with subfolders)
 - Built to be used together

Think of a **module** as one tool, and a **library** as a full set of tools for a **specific job**.

Python Library Structure - Demo

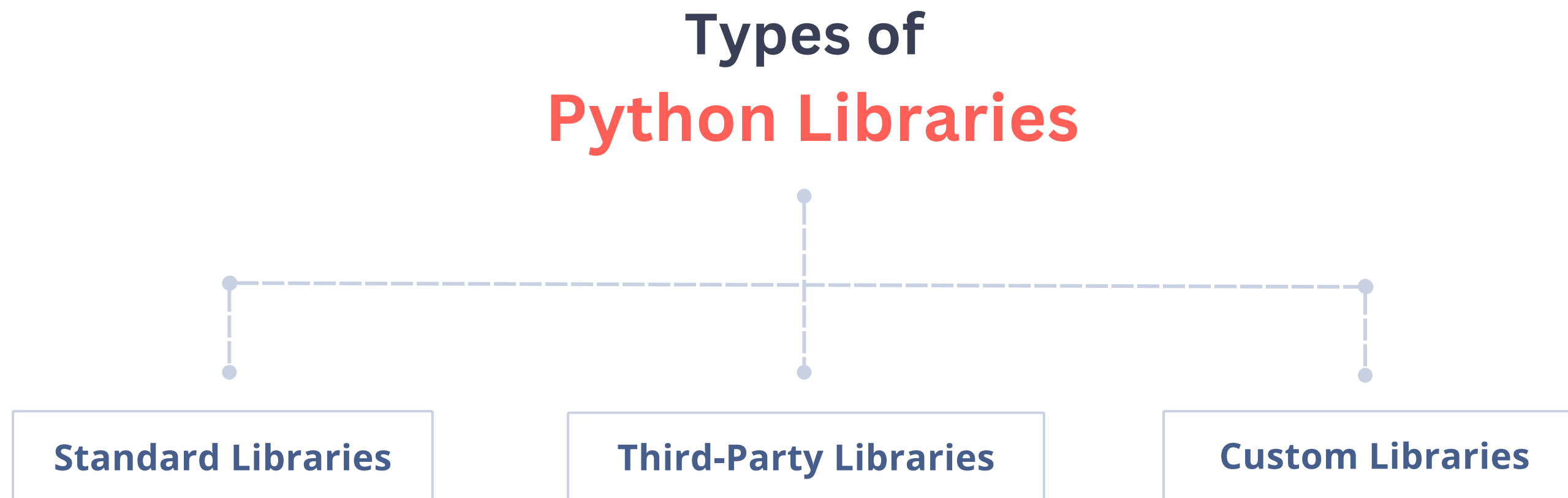


why is this structure important?

- It keeps your code **organized** and **reusable**.
- If we need a function from **module1.py**, we can import it into **script.py** and use it directly.

Types of Python Libraries

There are **three common types** of libraries you'll use in Python:

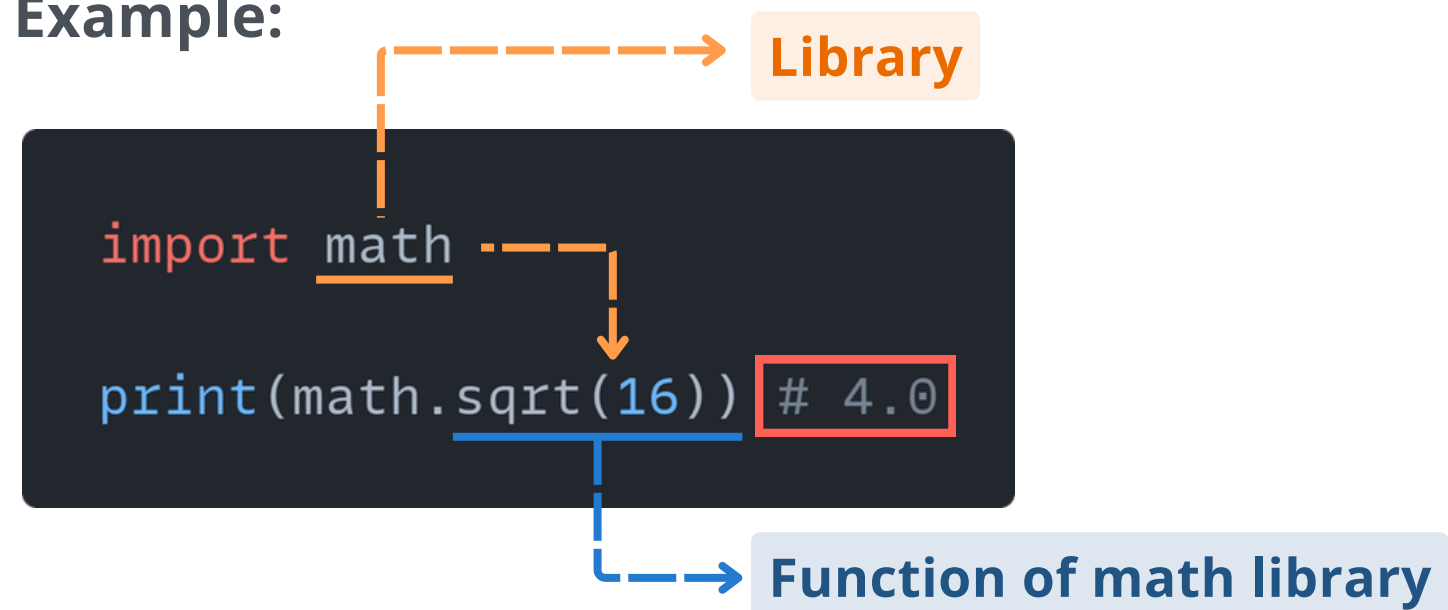


A. Standard Libraries

These are built into Python. You don't need to install anything. Just import and use them.

- **os** – for working with files and folders.
- **math** – for mathematical functions.
- **datetime** – for handling dates and times.

Example:



B. Third-Party Libraries

- These are **not included** with Python by default.
- We have to **install them** using **pip**
- These libraries allow you to extend Python's **capabilities** beyond the standard library.

Examples:

- **pandas** – used for data analysis.
- **Flask** – used for web development.
- **requests** – used for sending HTTP requests.

To install:

```
pip install pandas
```

downloads and installs pandas

any other libraries it depends on.

Then we can use it in your code:

```
import pandas as pd
```

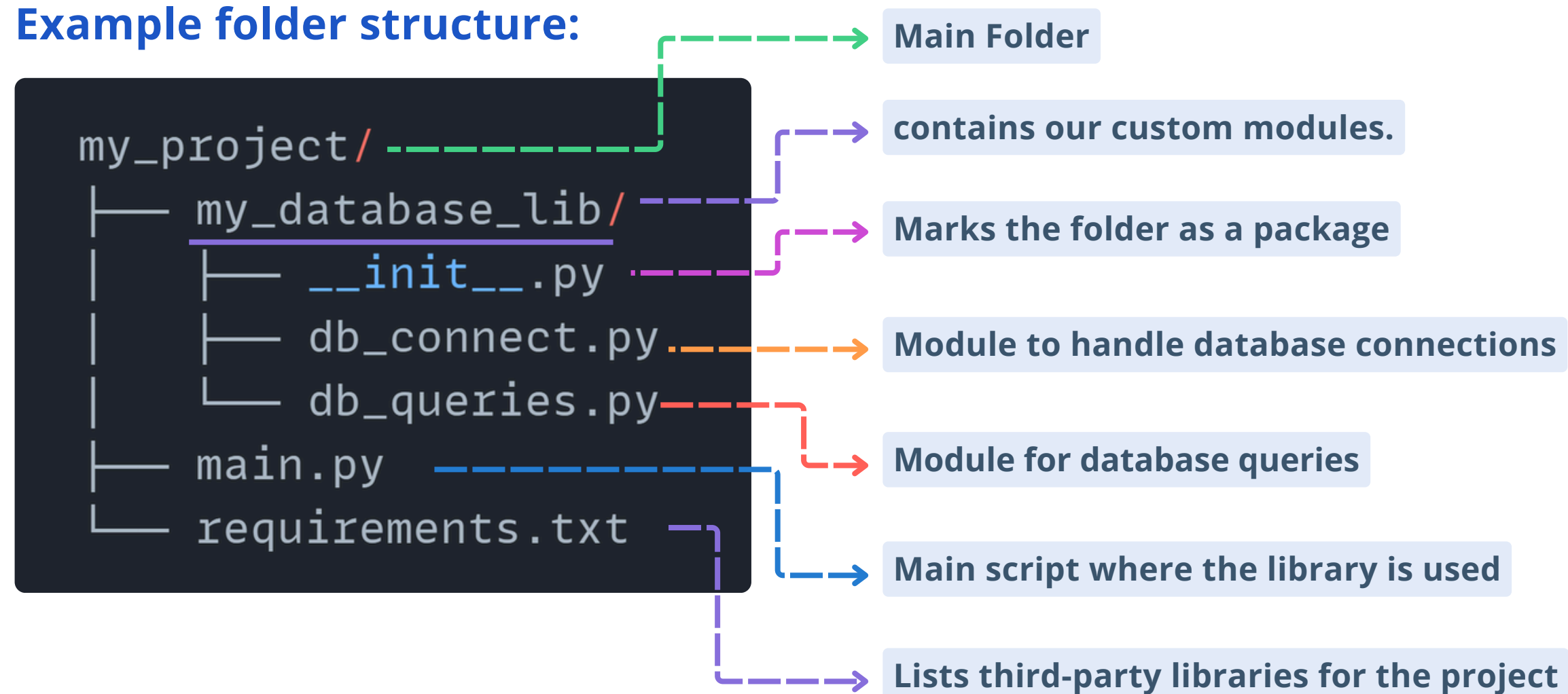
Alias

Libraries

C. Custom Libraries

- These are libraries **you** or **your team** create for a **specific project**.
- These are typically **collections** of **modules** that are designed to be **reusable** across **multiple projects**.

Example folder structure:



When Do We Need to Find a Library?

- When you're working on a task and think, “**Someone has probably done this before.**”
- Common needs might include:
 - Reading image files
 - Making HTTP requests
 - Automating tools
 - Handling large datasets

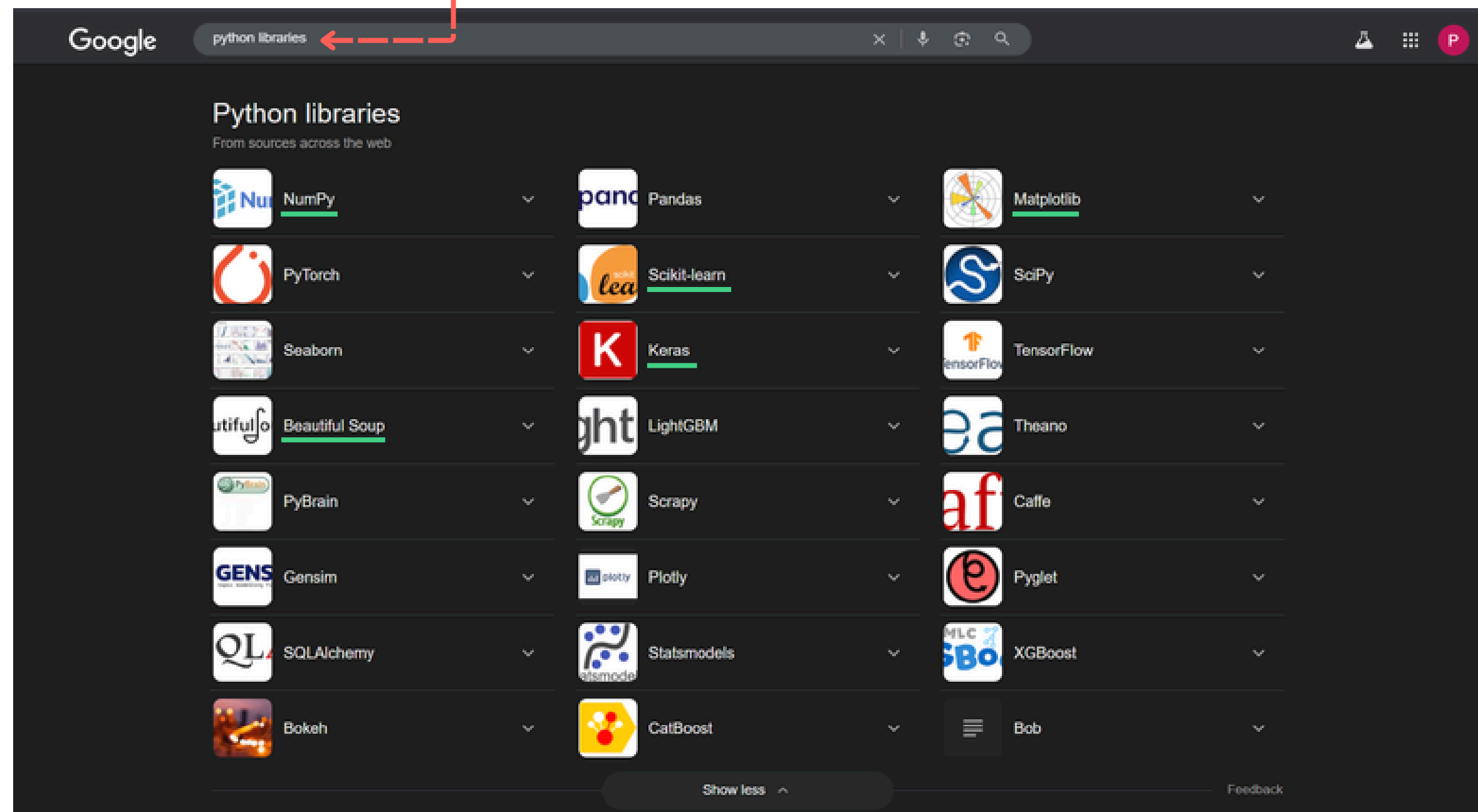
Rather than **build a solution from scratch**, it's a good idea to check if a **library already exists.**

Finding Python Libraries Online

If you need a library for a specific task, here are two ways to find one:

A. Searching for Libraries on Google - General Search

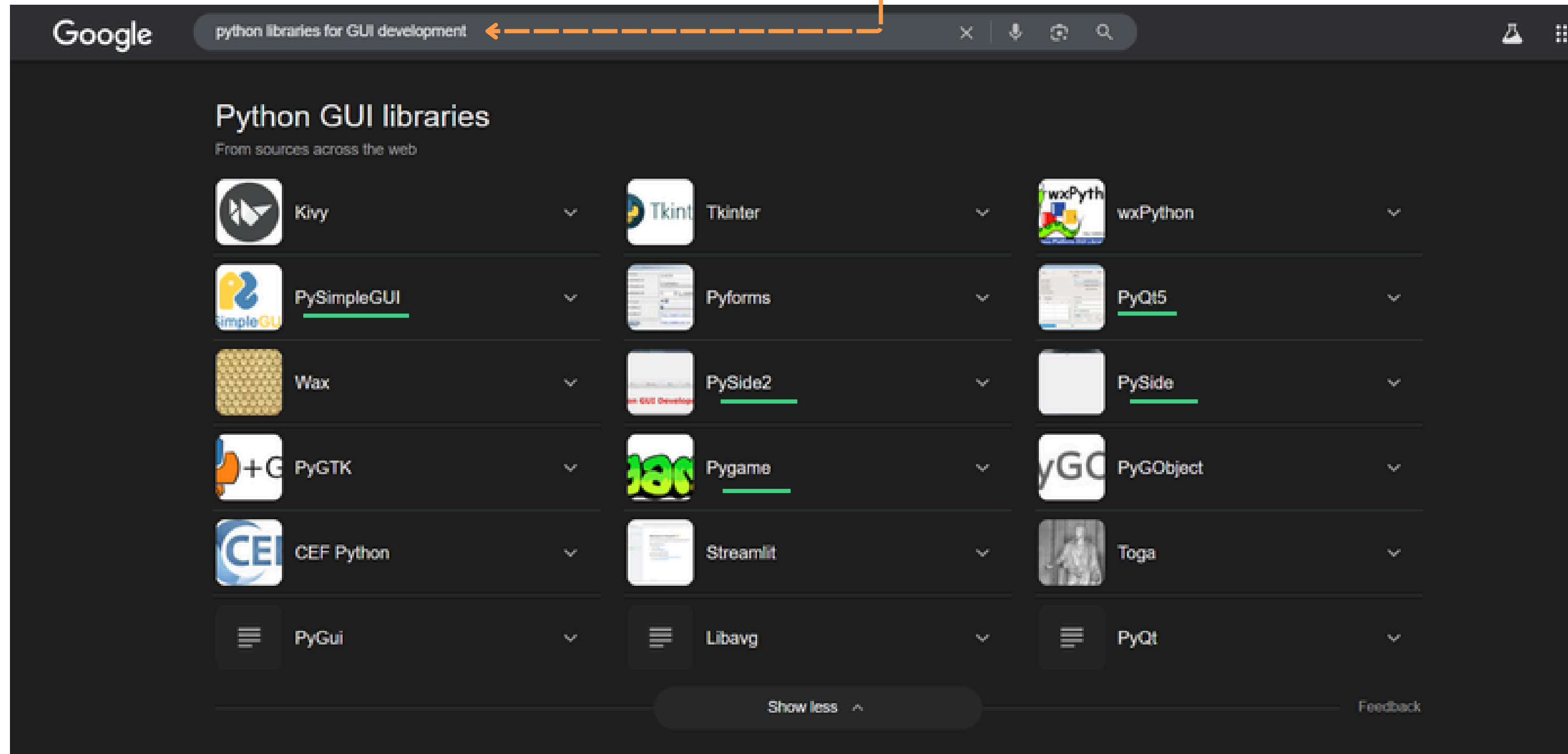
Simply  type "**Python libraries**" to explore a broad range of popular libraries.



B. Searching for Libraries on Google - Targeted Search

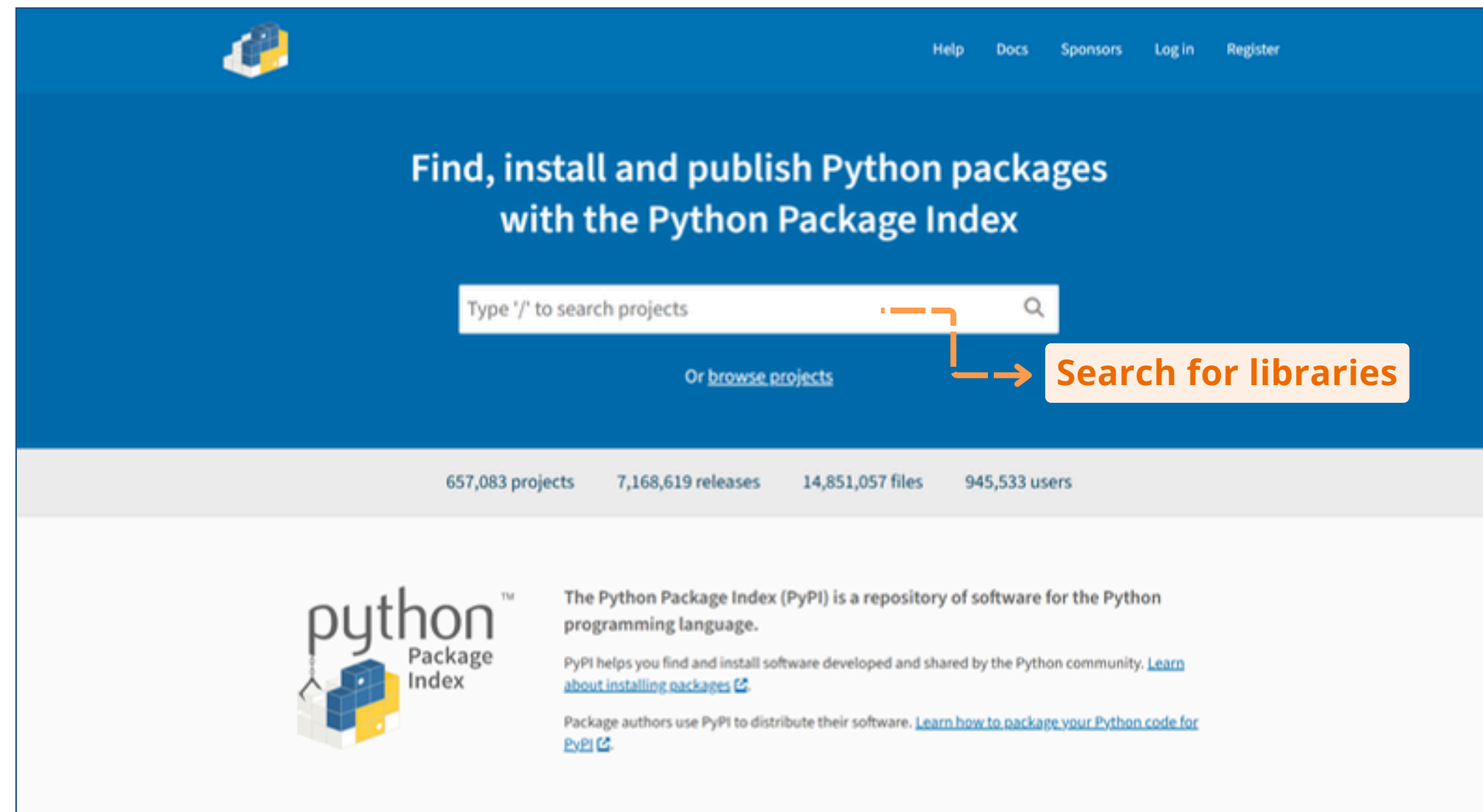
Focus your query on what you need:

👉 Search python libraries for GUI development



What is PyPI?

- PyPI stands for **Python Package Index**.
- It's the official online repository for Python **libraries** and **packages**.
- You can think of it as:
 - A **central library shelf** for Python
 - A place where developers **publish, share, and download** reusable code



How to Use PyPI:

- Visit [PyPI · The Python Package Index](#).
- Use the search bar to look up a library → Example: **pandas**, **Flask**, or **OpenEXR**

The screenshot shows the PyPI search results for the term 'panda'. The search bar at the top contains 'panda'. Below the search bar, there are filters on the left and a list of results. The first result, 'panda', is highlighted with a red box. An orange callout box with the text 'Click to See' points to this result. A dashed red line connects the search bar to the 'panda' result. A dashed orange line connects the 'Click to See' box to the list of results. A dashed purple line connects the list of results to a purple callout box on the right that says 'List of all libraries which has word panda common'.

Project Name	Description	Published
panda	A Python implementation of the Panda REST interface	Aug 3, 2015
airdrop-panda	一个关于web3 airdrop的python包	Feb 26, 2024
cameo-panda	This is a cameo_panda note	Apr 8, 2022
clean-panda	Data cleaner based on pandas	Jun 30, 2022
ndd-panda	Neuro Data Design EEG Preprocessing Pipeline	Jun 1, 2017
gym-panda	An OpenAI Gym Env for Panda	Jun 13, 2020

- Click on any library to view details such as:
 - *What the library does.*
 - *How to install it.*
 - *Links to its documentation or source code.*

List of all libraries which has word
panda common

Python bindings for the Qt cross-platform application and UI framework

Navigation

Project description

Release history

Download files

Verified details ✓

These details have been [verified by PyPI](#)

Owner

Qt Project

Maintainers



falt



QtForPython

Unverified details

These details have **not** been verified by PyPI

Project links

Changelog

Documentation

Homepage

Project description

PySide6

Introduction

Important: for Qt5 compatibility, check [PySide2](#)

PySide6 is the official Python module from the [Qt for Python project](#), which provides access to the complete Qt 6.0+ framework.

The Qt for Python project is developed in the open, with all facilities you'd expect from any modern OSS project such as all code in a git repository and an open design process. We welcome any contribution conforming to the [Qt Contribution Agreement](#).

Installation



Since the release of the [Technical Preview](#) it is possible to install via `pip`, both from Qt's servers and [PyPi](#):

```
pip install PySide6
```

Please note: this wheel is an alias to other two wheels [PySide6 Essentials](#) and [PySide6 Addons](#), which contains a predefined list of Qt Modules.

Dependencies

PySide6 versions following 6.0 use a C++ parser based on [Clang](#). The Clang library (C-bindings), version 13.0 or higher

Examples of Libraries You'll Find on PyPI:

Category	Libraries
• Data Analysis	→ pandas, numpy
• Web Development	→ Flask, Django
• Automation	→ selenium, pyautogui
• GUI Tools	→ PySide2, Tkinter
• VFX Specific	→ OpenEXR, imageio, OpenTimelineIO

Why Use PyPI?

Here's why PyPI is a favorite among Python developers:

- **Centralized Hub:** It's like an official store for Python libraries.
- **Free and Open-Source:** Most libraries are free and open for contributions.
- **Wide Variety:** From simple utilities to complex frameworks, PyPI covers almost every domain.

Understanding Default Python Libraries

- What Are Default Python Libraries?
- Using Default Libraries - os, sys, random, math

What Are Default Python Libraries?

When you install Python, it comes with a **built-in set of libraries** called the **Standard Library. or default Libraries**

- ✓ You don't need to install them separately.
- ✓ They're available as soon as Python is installed

Default libraries supports everyday needs like:

- Working with files and directories
- Doing math calculations
- Handling random values
- Accessing system-level information
- Managing dates and times

Demonstrating Default Libraries

Let's look at two of the most commonly **used default libraries**: **os** and **sys**.

```
import os
```

Helps us work with files, directories, and paths.

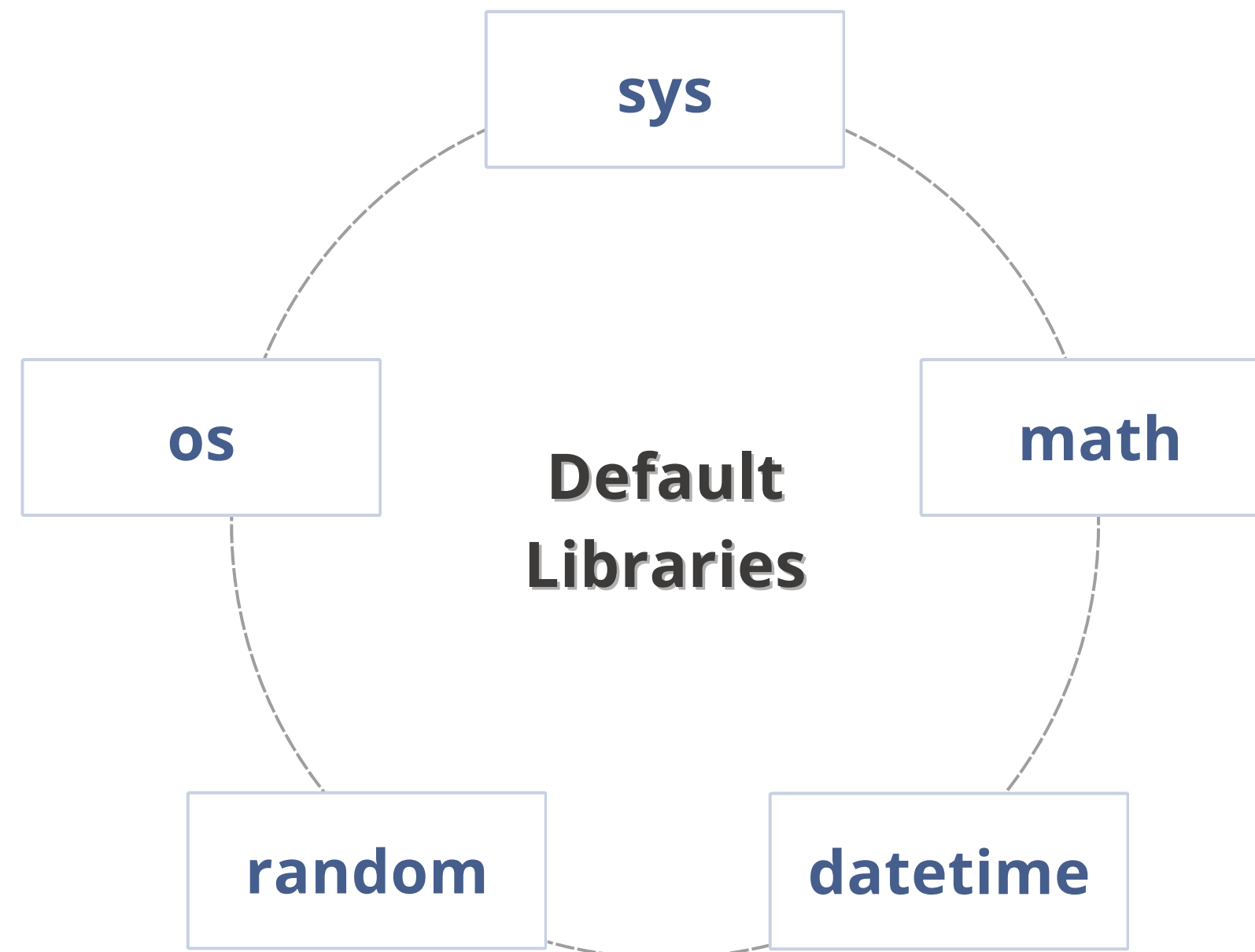
```
import sys
```

Gives you details about the system Python is running on

like version, platform, or script arguments

Using Default Python Libraries

GamutX



OS Module

```
import os
```

-----> module

```
# Printing the module itself
print(os)
```

```
# Output:
```

```
<module 'os' from 'D:\\Python3_9_7\\lib\\os.py'>
```

-----> location of it

```
# Listing all available functions and attributes in the os module
print(dir(os))
```

-----↑

```
# Output:
```

```
['DirEntry', 'F_OK', 'GenericAlias', 'Mapping', 'MutableMapping', 'O_APPEND', 'O_BINARY', 'O_CREAT', 'O_EXCL', 'O_NOINHERIT',
'O_RANDOM', 'O_RDONLY', 'O_RDWR', 'O_SEQUENTIAL', 'O_SHORT_LIVED', 'O_TEMPORARY', 'O_TEXT', 'O_TRUNC', 'O_WRONLY', 'P_DETACH',
'P_NOWAIT', 'P_NOWAITO', 'P_OVERLAY', 'P_WAIT', 'PathLike', 'R_OK', 'SEEK_CUR', 'SEEK_END', 'SEEK_SET', 'TMP_MAX', 'W_OK', 'X_OK',
'_AddedDllDirectory', '_Environ', '__all__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__',
'__package__', '__spec__', '_check_methods', '_execvpe', '_exists', '_exit', '_fspath', '_get_exports_list', '_walk', '_wrap_close',
'abc', 'abort', 'access', 'add_dll_directory', 'altsep', 'chdir', 'chmod', 'close', 'closerange', 'cpu_count', 'curdir', 'defpath',
'device_encoding', 'devnull', 'dup', 'dup2', 'environ', 'error', 'execl', 'execle', 'execlp', 'execlpe', 'execv', 'execve', 'execvp',
'execvpe', 'extsep', 'fdopen', 'fsdecode', 'fsencode', 'fspath', 'fstat', 'fsync', 'ftruncate', 'get_exec_path', 'get_handle_inheritable',
'get_inheritable', 'get_terminal_size', 'getcwd', 'getcwdb', 'getenv', 'getlogin', 'getpid', 'getppid', 'isatty', 'kill', 'linesep',
'link', 'listdir', 'lseek', 'lstat', 'makedirs', 'mkdir', 'name', 'open', 'pardir', 'path', 'pathsep', 'pipe', 'popen', 'putenv',
'read', 'readlink', 'remove', 'removedirs', 'rename', 'renames', 'replace', 'rmdir', 'scandir', 'sep', 'set_handle_inheritable',
'set_inheritable', 'spawnl', 'spawnle', 'spawnv', 'spawnve', 'st', 'startfile', 'stat', 'stat_result', 'statvfs_result', 'strerror',
'supports_bytes_environ', 'supports_dir_fd', 'supports_effective_ids', 'supports_fd', 'supports_follow_symlinks', 'symlink',
'sys', 'system', 'terminal_size', 'times', 'times_result', 'truncate', 'umask', 'uname_result', 'unlink', 'unsetenv',
'urandom', 'utime', 'waitpid', 'waitstatus_to_exitcode', 'walk', 'write']
```


sys Module

GamutX

`import sys` → module

`print(dir(sys))` → used to list all functions, class, attributes

Output:

```
['__breakpointhook__', '__displayhook__', '__doc__', '__excepthook__', '__interactivehook__', '__loader__', '__name__',
 '__package__', '__spec__', '__stderr__', '__stdin__', '__stdout__', '__unraisablehook__', '_base_executable', '_clear_type_cache',
 '_current_frames', '_debugmallocstats', '_enablelegacywindowsfsencoding', '_framework', '_getframe', '_git', '_home', '_xoptions',
 'addaudithook', 'api_version', 'argv', 'audit', 'base_exec_prefix', 'base_prefix', 'breakpointhook', 'builtin_module_names', 'byteorder',
 'call_tracing', 'copyright', 'displayhook', 'dllhandle', 'dont_write_bytecode', 'exc_info', 'excepthook', 'exec_prefix', 'executable', 'exit',
 'flags', 'float_info', 'float_repr_style', 'get_asyncgen_hooks', 'get_coroutine_origin_tracking_depth', 'getallocatedblocks', 'getdefaultencoding',
 'getfilesystemencodeerrors', 'getfilesystemencoding', 'getprofile', 'getrecursionlimit', 'getrefcount', 'getsireof', 'getswitchinterval', 'gettrace',
 'getwindowsversion', 'hash_info', 'hexversion', 'implementation', 'int_info', 'intern', 'is_finalizing', 'maxsize', 'maxunicode', 'meta_path', 'modules',
 'path', 'path_hooks', 'path_importer_cache', 'platform', 'platlibdir', 'prefix', 'pycache_prefix', 'set_asyncgen_hooks', 'set_coroutine_origin_tracking_depth',
 'setprofile', 'setrecursionlimit', 'setswitchinterval', 'settrace', 'stderr', 'stdin', 'stdout', 'thread_info', 'unraisablehook', 'version', 'version_info', 'warnoptions', 'winver']
```

`print(sys.path)` → list of paths Python searches for modules

Output:

```
['c:\\Users\\pralhad\\Desktop\\Work\\Academy\\gamutx-VSD-MASTER', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\python39.zip',
 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\DLLs', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib',
 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39', 'C:\\Users\\pralhad\\AppData\\Roaming\\Python\\Python39\\site-packages',
 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages\\win32',
 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages\\win32\\lib', 'C:\\Users\\pralhad\\AppData\\Local\\Programs\\Python\\Python39\\lib\\site-packages\\Pythonwin']
```

`print(sys.platform)` #Output: win32

`print(sys.version)` → tells us which operating system Python is currently running on.

Output:

3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (AMD64)]

Python Version

System Date and time

Operating System info

Math Module

```
import math → module
```

```
print(dir(math))
```

```
# Output:
```

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos',  
'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb',  
'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp',  
'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma',  
'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt',  
'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan',  
'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin',  
'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

```
print(math.pi)
```

Variable in math module

```
# Output: 3.141592653589793
```

```
print(math.tan(30))
```

Function in math module

```
# Output: -6.405331196646276
```

random Module

```
import random —→ module
```

```
print(dir(random))
```

```
# Output:
```

```
['BPF', 'LOG4', 'NV_MAGICCONST', 'RECIP_BPF', 'Random', 'SG_MAGICCONST', 'SystemRandom', 'TWOPI',  
 '_Sequence', '_Set', '__all__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',  
 '__name__', '__package__', '__spec__', '_accumulate', '_acos', '_bisect', '_ceil', '_cos', '_e',  
 '_exp', '_floor', '_inst', '_log', '_os', '_pi', '_random', '_repeat', '_sha512', '_sin', '_sqrt',  
 '_test', '_test_generator', '_urandom', '_warn', 'betavariate', 'choice', 'choices', 'expovariate',  
 'gammavariate', 'gauss', 'getrandbits', 'getstate', 'lognormvariate', 'normalvariate', 'paretovariate',  
 'randbytes', 'randint', 'random', 'randrange', 'sample', 'seed', 'setstate', 'shuffle', 'triangular',  
 'uniform', 'vonmisesvariate', 'weibullvariate']
```

```
print(random.random()) ———→ random function return random numbers from 0 to 1
```

```
# Output: 0.9949294041232459
```

Magic Methods in Python Library

- What Are Magic Methods?
- `__file__`
- `__name__`
- `__doc__`
- `__version__`
- Why Should We Care?

What Are Magic Methods?

In Python, **some variable names start and end with double underscores** like this:

- `__name__`
- `__file__`
- `__doc__`
- `__version__`

These are called **magic variables** also known as **dunder variables**.

They are:

- **Automatically created by Python**
- Already **attached** to **modules** or **scripts**
- Used to **store important information** (metadata)

__file__

__file__ is a built-in variable that holds the path to the current script file.

- It **exists by default** when Python runs a file
- It **gives** us the **full path as a string**

```
import os
```

```
print("Current :", __file__)
```

```
print("OS module :", os.__file__)
```

Shows Current Running File Location

Show Location Where OS Module Come From

```
# Current : C:\Users\pralhad\Desktop\demo\main.py
```

```
# OS module : C:\Python312\Lib\os.py
```

- **__file__** is only available when the script is run from a file,
- Can be used with `os.path` to manipulate paths.

__name__

__name__ is a built-in variable that **tells us whether the script is being run directly or being imported.**

```
import os
```

```
print( __name__ )  
print(os.__name__)
```

Tells If script is being run directly
or being imported.

```
# __main__  
# os
```

- When a **file is run directly**, __name__ is set to "**__main__**"
- If **imported as a module**, __name__ is set to the **module's name**

__doc__

It stores the documentation string (**docstring**) of a **module, function, or other Python object**.

Library-level __doc__

```
import os
```

```
print("OS Module Doc:", os.__doc__)
```

returns the documentation as a string

```
'''
```

```
# Output:
```

```
OS Module Doc: OS routines for NT or Posix depending on what system we're on.
```

```
This exports:
```

- all functions from posix or nt, e.g. unlink, stat, etc.
- os.path is either posixpath or ntpath
- os.name is either 'posix' or 'nt'
- os.curdir is a string representing the current directory (always '.')
- os.pardir is a string representing the parent directory (always '..')
- os.sep is the (or a most common) pathname separator ('/' or '\\')
- os.extsep is the extension separator (always '.')
- os.altsep is the alternate pathname separator (None or '/')
- os.pathsep is the component separator used in \$PATH etc
- os.linesep is the line separator in text files ('\r' or '\n' or '\r\n')
- os.defpath is the default search path for executables
- os.devnull is the file path of the null device ('/dev/null', etc.)

```
Programs that import and use 'os' stand a better chance of being portable between different platforms. Of course, they must then only use functions that are defined by all platforms (e.g., unlink and opendir), and leave all pathname manipulation to os.path (e.g., split and join).
```

```
'''
```


Library Function `__doc__`

```
import os
```

Module

```
print(os.listdir.__doc__)
```

Function

Dunder Variable

```
'''
```

```
# Output:
```

```
Return a list containing the names of the files in the directory.
```

```
path can be specified as either str, bytes, or a path-like object. If path is bytes,
the filenames returned will also be bytes; in all other circumstances
the filenames returned will be str.
```

```
If path is None, uses the path='.'.
```

```
On some platforms, path may also be specified as an open file descriptor;\
the file descriptor must refer to a directory.
```

```
If this functionality is unavailable, using it raises NotImplementedError.
```

```
The list is in arbitrary order. It does not include the special
entries '.' and '..' even if they are present in the directory.
```

```
'''
```

- Docstrings must be placed immediately after the function or module starts.
- They are written using triple quotes: `"""..."""`
- `__doc__` helps keep our code self-documenting and easy to maintain.
- **If there is no top-level docstring, `__doc__` will be None.**

__version__

__version__ is a **special variable that stores the version number of a module or library.**

- It shows the current version of the library.
- It is usually defined inside external or third-party modules.

```
import colorama
```

```
print(colorama.__version__)
```

```
# 0.4.6
```

Shows Version of Given Module

Important

- __version__ is not available in every built-in module.
- Many third-party libraries define __version__ to show their version number.
- If a **module does not define __version__, trying to access it will raise an error.**

Key Points

- **`__file__`** → Helps locate the current script's path
- **`__name__`** → Helps control code execution (run vs import)
- **`__doc__`** → Stores useful descriptions for scripts and functions
- **`__version__`** → Shows the version number of a module or library (if defined).

Working with Default Libraries

- os Library
- shutil Library
- shutil.copy() vs shutil.copy2()
- sys Library
- random Library
- math Library
- datetime Library
- Formatting Dates with strftime

os Library

The **os** module helps you interact with your computer's **operating system** through **Python**.

We can use it to:

- Get the current directory
- Create new folders
- Check if a file exists
- Work with paths in a way that's compatible across different operating systems

```
import os → module

current_directory = os.getcwd()
print(f"Current directory: {current_directory}") # Current directory: C:\Users\pralhad\Desktop\Work

new_directory = "my_folder"
os.mkdir(new_directory) → Creates a new folder named 'my_folder'
print(f"Created a new directory: {new_directory}") # Created a new directory: my_folder

file_name = "my_file.txt"
exists = os.path.exists(file_name) → Returns True if the file or folder exists
print(f"Does {file_name} exist? {exists}") # Does my_file.txt exist? False

print(os.sep) → Print OS-specific path separator # output : \
```


Why use os.sep?

Different operating systems use different path formats.

- **Windows:** uses **backslash **
- **macOS & Linux:** use **forward slash /**

Using **os.sep** helps you write code that works on any **operating system**, without needing to **hardcode** the **separator**.

```
import os

mac_path = "/Users/pralhad/Desktop/Work/tst/r.md"

cross_platform_path = mac_path.replace("/", os.sep)

print("Converted path:", cross_platform_path)
# Output:
# Converted path: \Users\pralhad\Desktop\Work\tst\r.md
```

macOS path

Replace / with OS-specific separator

shutil Library

- The **shutil module** is part of **Python's standard library**.
- It provides higher-level file operations, such as :
 - Copying files and folders
 - Moving and renaming items
 - Removing entire directory trees
 - Creating archive files (like .zip or .tar)
 - Getting disk usage stats

Copying a File

```
import shutil
```

```
shutil.copy("myfile.txt", "backup/")  
print("File copied.")
```

Copies file Content only

Which File

Destination

Before Copy

✓ backup

After Copy

✓ backup
myfile.txt

Copying a File with Metadata

```
import shutil

shutil.copy2("myfile.txt", "backup/")
print("File copied with metadata.")
```

- `shutil.copy2()` does the same as `copy()`
- but also **copies file metadata**, such as:
 - **Last modified time**
 - **Permissions**

Moving a File

```
import shutil

shutil.move("draft_version.mov", "final_renders/")
print("File moved.")
```

which one ↑

Destination ↑

- `shutil.move(src, dst)` moves a file or folder to a new location.
- If `dst` exists and is a directory, the file will be moved **inside** it.
- If the destination file already exists, it will be **overwritten**.

Deleting a Folder & All Its Contents

```
import shutil
import os

if os.path.exists("temp_cache"):

    shutil.rmtree("temp_cache")
    print("Temporary cache deleted.")

# Output
# Temporary cache deleted."
```

Check If Folder Exist or not

Deletes a Directory & Everything Inside It

- **Caution** → this is a **permanent delete**.

- Often used to clean up simulation caches or temp folders after rendering.

Archiving a Folder (Creating a .zip)

```
import shutil

shutil.make_archive("backup_assets", "zip", "assets")

print("Archive created: backup_assets.zip")

# Output :
# Archive created: backup_assets.zip
```

Output Name **Format** **Folder to Archive**

↑ ↑ ↑

creates a compressed archive.

- "backup_assets" is the output name (without .zip)
- "zip" is the format (zip, tar, etc.)
- "assets" is the folder to archive
- Used when backing up shot folders or creating delivery packages.

Checking Disk Usage

```
import shutil

usage = shutil.disk_usage("C:/")

print(f"Total: {usage.total}")
print(f"Used: {usage.used}")
print(f"Free: {usage.free}")
print()

total = usage.total / (1024 ** 3)
used = usage.used / (1024 ** 3)
free = usage.free / (1024 ** 3)

print(f"Total: {total:.2f} GB")
print(f"Used: {used:.2f} GB")
print(f"Free: {free:.2f} GB")
```

returns disk statistics for a given path.

Total disk space available

Total space used

Total free space

Converts bytes into GB

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PO

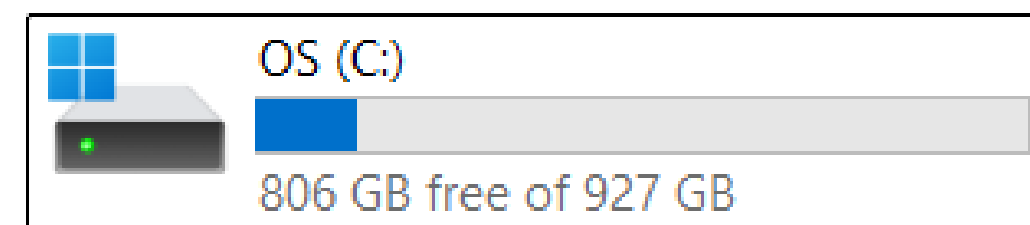
PS C:\Users\pralhad\Desktop\Cohort-EC1> & "C:/Program

• Total: 995727052800
  Used: 146720268288
  Free: 849006784512 } bytes

Total: 927.34 GB
Used: 136.64 GB
Free: 790.70 GB
PS C:\Users\pralhad\Desktop\Cohort-EC1> |
```

Useful before running heavy simulations or render jobs.

Devices and drives



shutil.copy() vs shutil.copy2()

Let's say we have a file named **original.txt** that already exists.

Checking the Last Modified Time of a File

```
import os

modified_time = os.path.getmtime("original.txt")

print("Last modified time:", modified_time)

# Last modified time: 1768486759.45046
```

Returns the last modified time of the file at the given path.

- value is returned as a floating-point number.

1768486759.45046 -----> *Represent Seconds*

*Number of full seconds
since Jan 1, 1970* *Fraction of a second
(450 milliseconds)*

Copy Using copy() and copy2()

```
import shutil
import os

shutil.copy("original.txt", "copy_file.txt")
shutil.copy2("original.txt", "copy2_file.txt")

original_time = os.path.getmtime("original.txt")
copy_time = os.path.getmtime("copy_file.txt")
copy2_time = os.path.getmtime("copy2_file.txt")

print("Original:", original_time)
print("copy(): ", copy_time)
print("copy2(): ", copy2_time)
```

creates a new file without Metadata

Create File while preserves the original file's metadata

returns the last modified time

Output

```
Original: 1768486759.45046
copy():  1768487820.515553
copy2(): 1768486759.45046
```

Difference

What This Proves

Method	Content Copied	Metadata Copied
• shutil.copy()	→ Yes	→ No
• shutil.copy2()	→ Yes	→ Yes

Summary

Task

Function Used

- | | | |
|-----------------------------------|---|------------------------------------|
| • Copying asset files | → | <code>shutil.copy()</code> |
| • Copying with metadata | → | <code>shutil.copy2()</code> |
| • Moving files or folders | → | <code>shutil.move()</code> |
| • Deleting a folder with contents | → | <code>shutil.rmtree()</code> |
| • Creating a zip archive | → | <code>shutil.make_archive()</code> |
| • Checking available disk space | → | <code>shutil.disk_usage()</code> |

sys Library

The **sys** module provides tools to interact with the **Python interpreter** and the **system**.

Common uses include:

- Reading command-line arguments
- Checking the version of Python
- Exiting the program

Get Command line Argument

```
import sys
args = sys.argv
print(f"Command-line arguments: {args}")
```

List of arguments passed to the script

```
PS C:\Users\pralhad\Desktop\merge_confict_demo> python t.py test_input.txt
Command-line arguments: ['t.py', 'test_input.txt']
PS C:\Users\pralhad\Desktop\merge_confict_demo>
```

Name of Script

Arguments we passed

Check Python Version

```
import sys
python_version = sys.version
print(f"Python version: {python_version}")
# Python version: 3.9.13 (tags/v3.9.13:6de2ca5, May 17 2022, 16:36:42) [MSC v.1929 64 bit (AMD64)]
```

Returns Python version

OS Deatils

Python Version

Microsoft C Compiler version

random Library

- The random module is **used to generate random numbers and make random choices**.
- It's useful in games, simulations, and testing.

```
import random
```

Module

Function

```
random_number = random.randint(1, 10)
print(f"Random number between 1 and 10: {random_number}")
# Random number between 1 and 10: 7
```

Generate a random number between 1 and 10

```
my_list = ["apple", "banana", "cherry"]
random_item = random.choice(my_list)
print(f"Random item from the list: {random_item}")
# Random item from the list: apple
```

Picks one item randomly

Shuffle a list randomly

```
random.shuffle(my_list)
print(f"Shuffled list: {my_list}")
# Shuffled list: ['apple', 'cherry', 'banana']
```

```
deck = ["ace", "two", "three", "four"]
random.shuffle(deck)
print(deck) # ['three', 'four', 'ace', 'two']
```


math Library

The **math** module provides built-in functions to perform **common mathematical operations**. like

- Common mathematical functions like **square root, sine, cosine**
- Useful constants like **π (pi)**
- Functions for **angles, rounding, powers, logs**, and more

```
import math
```

```
number = 16
```

```
sqrt = math.sqrt(number)
```

```
print(f"Square root of {number}: {sqrt}")
```

```
# Square root of 16: 4.0
```

→ Gives square root of a number

```
pi = math.pi → constant pi having pi value
```

```
print(f"Value of pi: {pi}")
```

```
# Value of pi: 3.141592653589793
```

```
angle = math.pi / 4
```

```
sine = math.sin(angle)
```

→ Provide sin of given angle

```
print(f"Sine of {angle} radians: {sine}")
```

```
# Sine of 0.7853981633974483 radians: 0.7071067811865476
```

datetime Library

- The **datetime** module is part of **Python's standard library**. It helps you work with **dates** and **times**
- useful for **logging**, **scheduling**, **tracking changes**, and more.
- You can:
 - Get the **current date/time**
 - Create **specific dates**
 - Calculate the **difference between dates**
 - Format **dates for display**

```
import datetime
now = datetime.datetime.now()
print(f"Current date and time: {now}")
# Current date and time: 2025-07-18 14:48:47.648353

date_obj = datetime.date(2024, 5, 7)
print(f"Specific date: {date_obj}")
# Specific date: 2024-05-07

date1 = datetime.date(2024, 5, 1)
date2 = datetime.date(2024, 5, 7)
delta = date2 - date1
print(f"Days between {date1} and {date2}: {delta.days}")
# Days between 2024-05-01 and 2024-05-07: 6
```

Class (points to `datetime.datetime`)

use to get the current date and time (points to `datetime.datetime.now()`)

use to Create Date Object (points to `datetime.date(2024, 5, 7)`)

(2024 , 5 , 7) (points to the arguments of `datetime.date()`)

year (points to 2024), **Month** (points to 5), **Day** (points to 7)

attribute that gives us the difference in full days. (points to `delta.days`)

Formatting Dates with strftime

- **strftime()** stands for **string format time**.
- It lets you **format** a **date object** into a **string**, using a **specific layout** (e.g., **DD/MM/YY**, **YYYY-MM-DD**, etc.).

The diagram illustrates the process of formatting a date using the `datetime` module and the `date` class. It shows two examples of formatting a date object.

```
import datetime
date_obj = datetime.date.today()
formatted_date = date_obj.strftime("%d/%m/%y")
print(formatted_date)
# Output: 18/07/25

formatted = date_obj.strftime("%Y-%m-%d")
print(formatted)
# Output: 2025-07-18
```

Module (green box) points to `datetime`.
Class (purple box) points to `date`.
Returns today's date (orange box) points to `date_obj = datetime.date.today()`.
Day of the month (orange box) points to `%d` in the format string.
Month (purple box) points to `%m` in the format string.
Year (last two digits) (blue box) points to `%y` in the format string.
Format the date as YYYY-MM-DD (red box) points to the format string `"%Y-%m-%d"`.
Full year (white box) points to `%Y` in the format string.

Common strftime() Format Codes

Code	Description	Example
• %d	→ Day of the month (01–31)	→ 4
• %m	→ Month (01–12)	→ 12
• %y	→ Year (last 2 digits)	→ 24
• %Y	→ Full year	→ 2024
• %A	→ Weekday name	→ Wednesday
• %B	→ Month name	→ December

Use Case in Industry

- datetime Module - Use Case in Industry
- random Module - Use Case in Industry
- math Module - Use Case in Industry

datetime Module - Use Case in Industry

Managing Files and Logs:

- VFX work involves **constant file updates**: renders, caches, simulation outputs, scripts, etc.
- To **avoid overwriting**, teams use **timestamps** in **file names**.

Examples of file names:

- **shot001_v01_2024-12-04.nk** — Nuke script saved on Dec 4, 2024
- **simulation_cache_04-12-24_v02.abc** — Cache file with a clear version and date

Render logs:

```
[2024-12-04 14:30] Render started  
[2024-12-04 14:45] Frame 001 completed  
[2024-12-04 15:00] Render completed
```

These logs help **track progress** and **troubleshoot issues**.

random Module - Use Case in Industry

Procedural Generation and Simulation:

- Real-world **motion** and **surfaces** have **variation**.
- In VFX, we use **randomness** to make **digital elements** look **less perfect** and **more natural**.

Examples:

- **Particle FX** → Vary speed, direction, size of particles
- **Textures** → Random scratches, dirt, or rust
- **Camera Shake** → Add subtle random motion for realism
- **Leaf simulation:**
 - Randomize position and **fall speed**
 - Apply **slight** rotations

How random helps:

- **random.randint()** → Generate random **numbers**
- **random.choice()** → Pick random **items** (e.g., **animation offsets**)
- **random.shuffle()** → Randomize **order** of a **list**

Randomizing Leaf Fall

```
import random

x = random.uniform(-10, 10)
speed = random.uniform(1, 3)

print(f"Leaf at x: {x:.2f}, speed: {speed:.2f}")
# Leaf at x: 4.36, speed: 1.87
```

Random horizontal position

Random fall speed

uniform(a, b) → Returns a random floating-point number between **a** and **b**.

Real-world Impact:

- Makes simulations look **more organic**.
- Useful in **FX, layout**, and **environment work**.
- Frequently used in tool **scripts** for **scattering, variation**, and **presets**.

math Module - Use Case in Industry

Advanced Calculations

- Much of VFX involves math behind the scenes — for **accurate movements, curves, and rotations**.

Examples in VFX:

- **Geometric transformations** — Calculate **positions** using **angles** or **distances**
- **Circular animation** — Use `math.pi`, `math.sin()`, `math.cos()` to move something in a **circle**
- **Physics-based animation** — Trigonometric functions help **simulate movement** or **rotation**

VFX Example

```
import math

frame = 42
amplitude = 5
speed = 0.1

y = amplitude * math.sin(frame * speed)

print(f"Y position at frame {frame}: {y:.2f}")
# Y position at frame 42: -4.36
```

Diagram annotations:

- `frame = 42` is annotated with "Frame number".
- `amplitude = 5` is annotated with "Max height".
- `speed = 0.1` is annotated with "Oscillation speed".
- The expression `amplitude * math.sin(frame * speed)` is annotated with "Calculate oscillating Y position".

How math helps:

- `math.sin()`, `math.cos()` → Control **direction** or **oscillation**
- `math.pi` → Use exact values in **calculations**
- `math.sqrt()`, `math.atan2()` → Support **vector math**

Real-world Impact:

- Used in **rigging tools**, **simulation systems**, **animation helpers**.
- Essential for any tool that deals with **movement** or **space**.

Third-Party Libraries

- What Are Third-Party Libraries?
- Examples of Popular Third-Party Libraries
- Where to Find Third-Party Libraries
- Understanding the Package Page
- Key Sections on a PyPI Package Page

What Are Third-Party Libraries?

- In Python, **third-party libraries** are packages created by **the community**, not included with Python by **default**.
- You **install** them separately — they are not part of the **standard library**.

✓ You'll find third-party libraries for:

- *Web development*
- *Data analysis*
- *Image and video processing*
- *Machine learning*
- *Automation and scripting*
- *GUI development for tools (e.g., PySide2 for VFX)*

Examples of Popular Third-Party Libraries

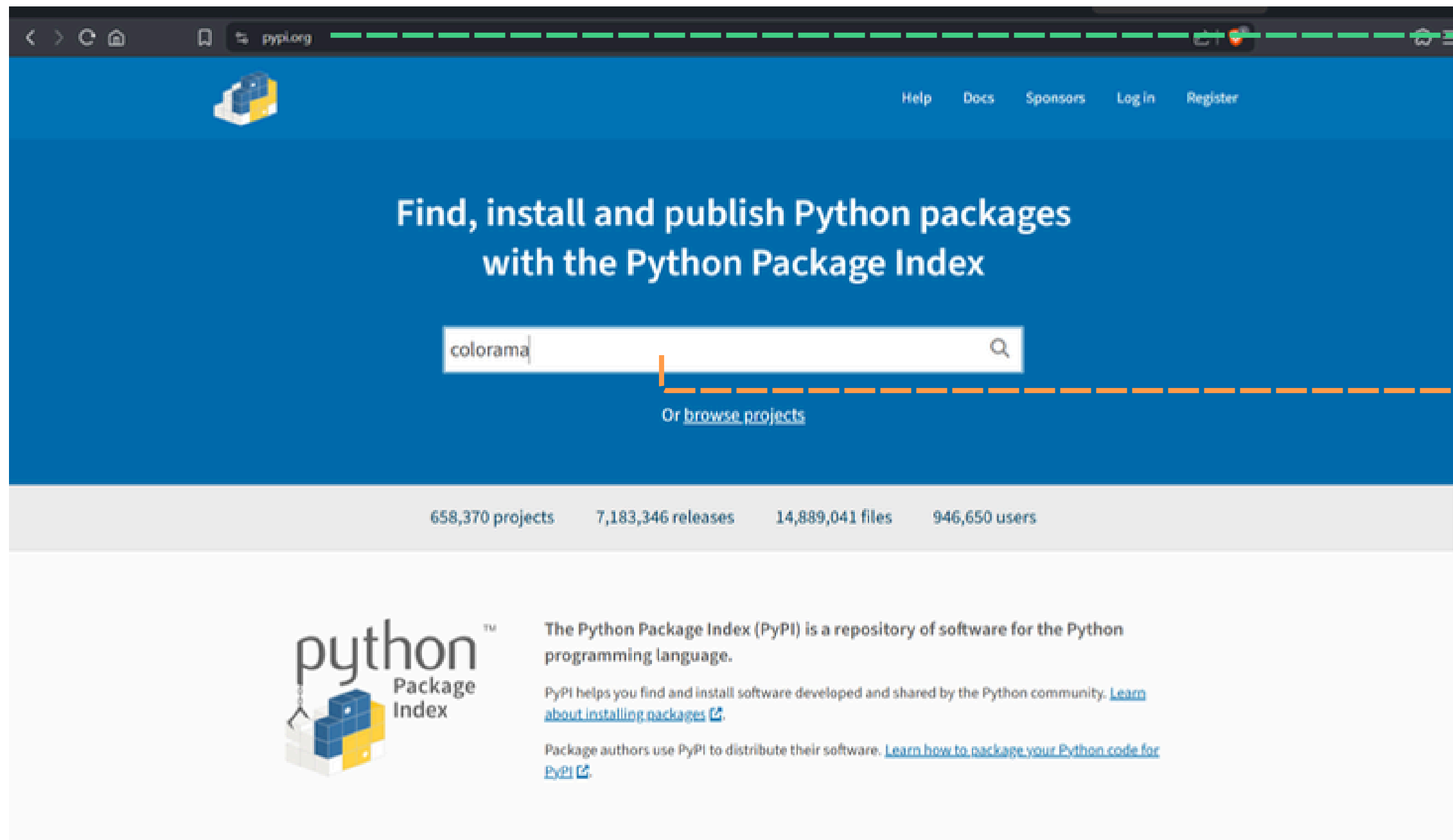
Code	Example
• colorama	→ Add color to terminal output
• pandas	→ Work with data tables (rows/columns)
• requests	→ Make HTTP requests (e.g., talk to APIs)
• OpenEXR	→ Read/write EXR image files (VFX workflows)
• PySide2	→ Build custom GUI tools for artists

Many of these packages are open-source and available on platforms like **PyPI** (Python Package Index).

Where to Find Third-Party Libraries

- To get started with **third-party libraries**, we need to know where to find them.
- The main place to search for **Python libraries** is **PyPI (Python Package Index)**. **PyPI** is like a **APP Store** for **Python libraries**.

How to Search for Libraries on PyPI:



• Visit: <https://pypi.org>

• Click: The top result named **colorama**

Understanding the Package Page on PyPI

When you open the page for a package like colorama, you'll see important details such as:

The screenshot shows the PyPI package page for **colorama** version **0.4.6**. The page has a blue header with a search bar, navigation links (Help, Sponsors, Log in, Register), and the package name and version. Below the header, there's a green box with a checkmark and 'Latest version', and a red box with 'Released: Oct 25, 2022'. A green box with 'pip install colorama' is also present. The main content area is divided into a 'Navigation' sidebar and a 'Project description' section. The 'Project description' section includes a 'Colorama' title, a description of the package, and a 'Usage Section' with a 'Description' box. The 'Usage Section' contains the text 'What the package does' and 'How to use it'. The 'Description' box is highlighted with a purple bracket and a dashed arrow pointing to it from the right. The 'Usage Section' is also highlighted with a purple bracket and a dashed arrow pointing to it from the right.

Package name: **colorama**

Package Version: **0.4.6**

Installation command: `pip install colorama`

Release Date: **Released: Oct 25, 2022**

Project description: **Colorama**

What the package does

Usage Section: **How to use it**

Key Sections on a PyPI Package Page

Section

What It Tells You

- **Name** → The name of the package (colorama)
- **Version** → The current version available (e.g., 0.4.6)
- **Description** → A short summary of what the package does
- **Installation** → How to install it using pip
- **Release History** → Older versions of the package and their release dates
- **Project Links** → Links to documentation, source code, bug tracker, etc.
- **License** → The type of license (e.g., MIT, Apache 2.0) useful for commercial project use

Package Manager - pip

- What is pip?
- How to Check if pip is Installed
- If pip Not Found
- How to Install a Package Using pip
- Where does pip Installs Packages?

What is pip?

- **pip** is the **official package manager** for **Python**.
- It lets us **install third-party** libraries (also called **packages**) from the internet from **PyPI**
- The name stands for:
 - 🖱️ **"Pip Installs Packages"**

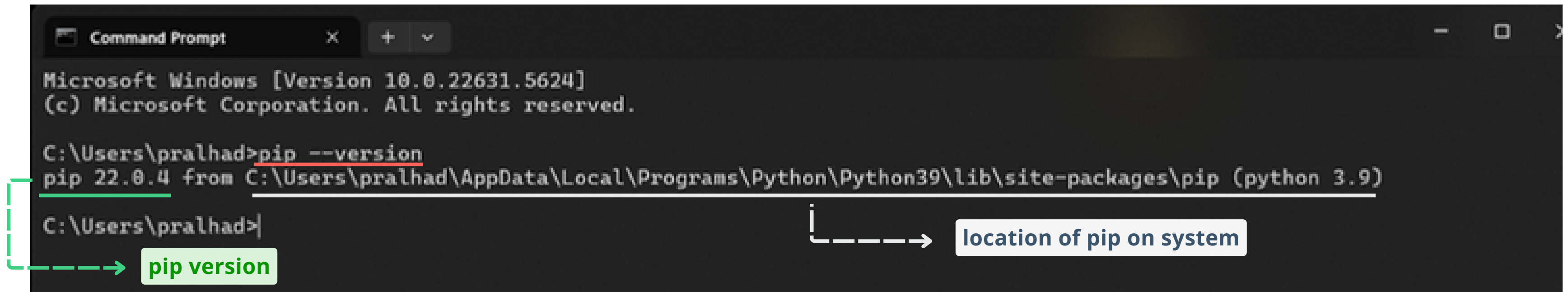
We'll use pip to install libraries that are not included with Python by default, such as:

- pandas (for data analysis)
- requests (for making web requests)
- colorama (for colored terminal text)

How to Check if pip is Installed

Before we **install any libraries**, let's check if **pip** is installed on **your system**. To do this:

- Open your **terminal (macOS/Linux)** or **Command Prompt (Windows)**.
- Type the following command:



```
Command Prompt
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>pip --version
pip 22.0.4 from C:\Users\pralhad\AppData\Local\Programs\Python\Python39\lib\site-packages\pip (python 3.9)

C:\Users\pralhad>
```

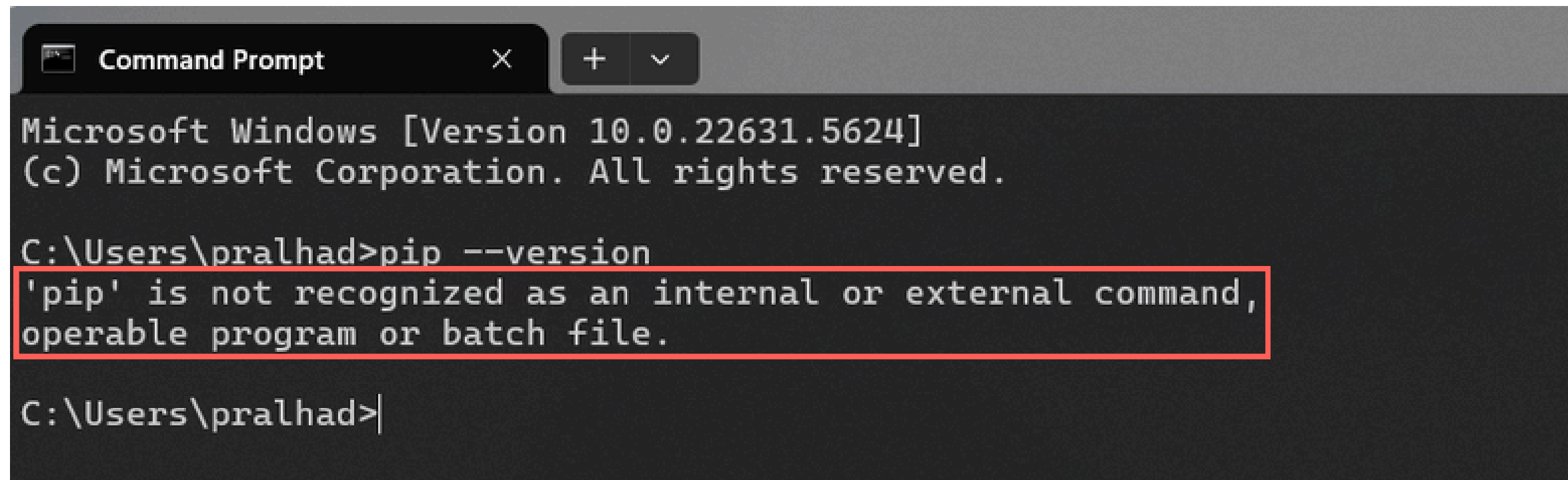
pip version

location of pip on system

If **pip** is **not installed**, you'll need to **install it first**. You can follow the official guide [here](#) to set it up.

If pip Not Found

- If you tried:



```
Command Prompt
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>pip --version
'pip' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\pralhad>
```

→ Then it likely means:

- pip **isn't installed**, or
- It's **not** correctly added to your **system's PATH**

Two ways to **Install pip**

Using ensurepip

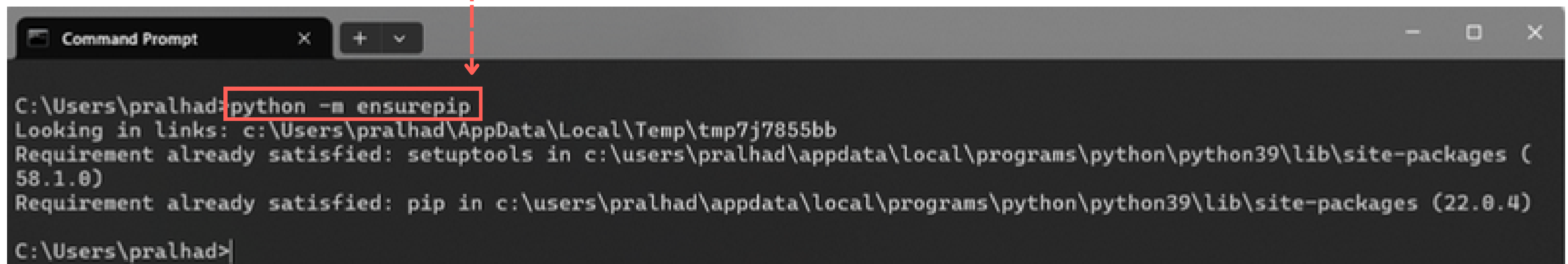
Using get-pip.py
Manual Installation

Method 1: Using ensurepip

If your **Python installation** supports it, this is the **easiest way**.

Steps

- Open your **terminal** or **command prompt**.
- Run this **command**:
 - `python -m ensurepip`



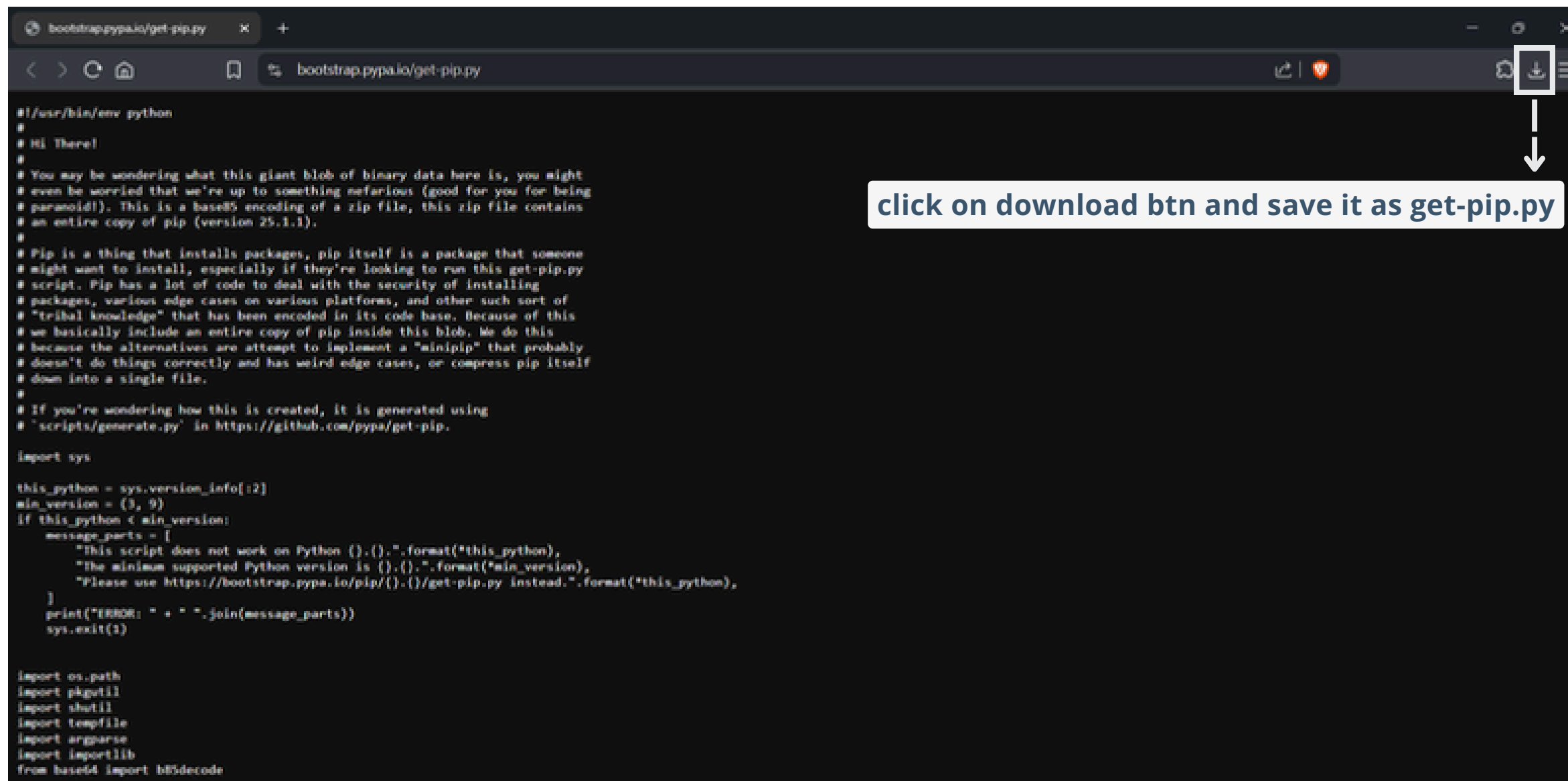
The screenshot shows a Windows Command Prompt window with the title bar 'Command Prompt'. The command prompt shows the user's current directory as 'C:\Users\pralhad' and the command 'python -m ensurepip' has been entered and executed. The output of the command is displayed below the command line, showing the paths where Python is looking for links and the status of the 'setuptools' and 'pip' packages. A red dashed arrow points from the command 'python -m ensurepip' in the list of steps to the same command in the Command Prompt window. The command 'python -m ensurepip' is also highlighted with a red box in the Command Prompt window.

```
C:\Users\pralhad>python -m ensurepip
Looking in links: c:\Users\pralhad\AppData\Local\Temp\tmp7j7855bb
Requirement already satisfied: setuptools in c:\users\pralhad\AppData\Local\programs\python\python39\lib\site-packages (58.1.0)
Requirement already satisfied: pip in c:\users\pralhad\AppData\Local\programs\python\python39\lib\site-packages (22.0.4)
C:\Users\pralhad>
```

This **tells** Python to set up **pip automatically** if the bundled **installer** is **available**.

Method 2: Using get-pip.py (Manual Installation)

Use this if **ensurepip** doesn't work on your system.



The screenshot shows a web browser window with the address bar displaying `bootstrap.pypa.io/get-pip.py`. The page content is a Python script. A white box with a black border highlights the download button in the top right corner of the browser window. A white arrow points from this box to a text box that says "click on download btn and save it as get-pip.py".

```
#!/usr/bin/env python
#
# Hi There!
#
# You may be wondering what this giant blob of binary data here is, you might
# even be worried that we're up to something nefarious (good for you for being
# paranoid!). This is a base85 encoding of a zip file, this zip file contains
# an entire copy of pip (version 25.1.1).
#
# Pip is a thing that installs packages, pip itself is a package that someone
# might want to install, especially if they're looking to run this get-pip.py
# script. Pip has a lot of code to deal with the security of installing
# packages, various edge cases on various platforms, and other such sort of
# "tribal knowledge" that has been encoded in its code base. Because of this
# we basically include an entire copy of pip inside this blob. We do this
# because the alternatives are attempt to implement a "minipip" that probably
# doesn't do things correctly and has weird edge cases, or compress pip itself
# down into a single file.
#
# If you're wondering how this is created, it is generated using
# 'scripts/generate.py' in https://github.com/pypa/get-pip.

import sys

this_python = sys.version_info[:2]
min_version = (3, 9)
if this_python < min_version:
    message_parts = [
        "This script does not work on Python {}.{}.".format(*this_python),
        "The minimum supported Python version is {}.{}.".format(*min_version),
        "Please use https://bootstrap.pypa.io/pip/{}.{}//get-pip.py instead.".format(*this_python),
    ]
    print("ERROR: " + " ".join(message_parts))
    sys.exit(1)

import os.path
import pkgutil
import shutil
import tempfile
import argparse
import importlib
from base64 import b85decode
```

Steps

- Download the installer script:
 - Go to the **official page**:
 - 🖱️ <https://bootstrap.pypa.io/get-pip.py>
 - Save the file as **get-pip.py**
- Open your terminal in the folder where you saved the script.
- Run the **script** with **Python**:
 - **python get-pip.py**
- **Confirm** → **pip --version**

This will **download** and **install** the **latest version of pip**.

Official pip – Documentation

GamutX

You can visit the official site for latest installation updates <https://pip.pypa.io/en/stable/installation/>

The screenshot shows the official pip documentation website for version 25.1.1. The left sidebar contains a navigation menu with links to 'Getting Started', 'Installation' (highlighted), 'User Guide', 'Topic Guides', 'Reference', 'Commands', 'PROJECT', 'Development', 'UX Research & Design', 'Changelog', 'Code of Conduct', and 'GitHub'. The main content area is titled 'Installation' and includes a sub-header 'Usually, pip is automatically installed if you are:' followed by a list of conditions: working in a virtual environment, using Python from python.org, and using Python not modified by a redistributor to remove ensurepip. Below this is the 'Supported Methods' section, which states that if pip is not installed, there are two mechanisms: ensurepip and get-pip.py. The 'ensurepip' section explains that Python includes an ensurepip module and provides a code block for Windows: `C:\> py -m ensurepip --upgrade`. The 'get-pip.py' section describes it as a bootstrapping script and provides a list of steps to download and run it. A right sidebar titled 'ON THIS PAGE' lists links to 'Supported Methods', 'Alternative Methods', 'Upgrading via', and 'Compatibility'. At the bottom right, there is a dropdown menu showing 'stable'.

pip documentation v25.1.1

Search

Getting Started

Installation

User Guide

Topic Guides

Reference

Commands

PROJECT

Development

UX Research & Design

Changelog

Code of Conduct

GitHub

Installation

Usually, pip is automatically installed if you are:

- working in a [virtual environment](#)
- using Python downloaded from [python.org](#)
- using Python that has not been modified by a redistributor to remove `ensurepip`

Supported Methods

If your Python environment does not have pip installed, there are 2 mechanisms to install pip supported directly by pip's maintainers:

- `ensurepip`
- `get-pip.py`

ensurepip

Python comes with an `ensurepip` module^[1], which can install pip in a Python environment.

Linux MacOS **Windows**

```
C:\> py -m ensurepip --upgrade
```

More details about how `ensurepip` works and how it can be used, is available in the standard library documentation.

get-pip.py

This is a Python script that uses some bootstrapping logic to install pip.

- Download the script, from <https://bootstrap.pypa.io/get-pip.py>.
- Open a terminal/command prompt, `cd` to the folder containing the `get-pip.py` file and run:

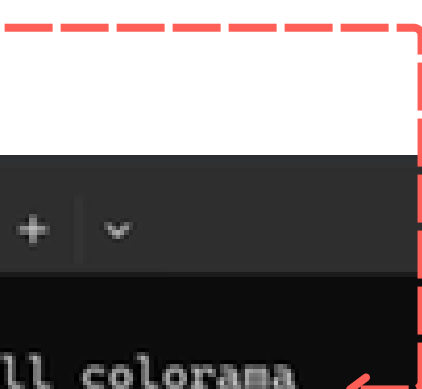
Linux MacOS **Windows**

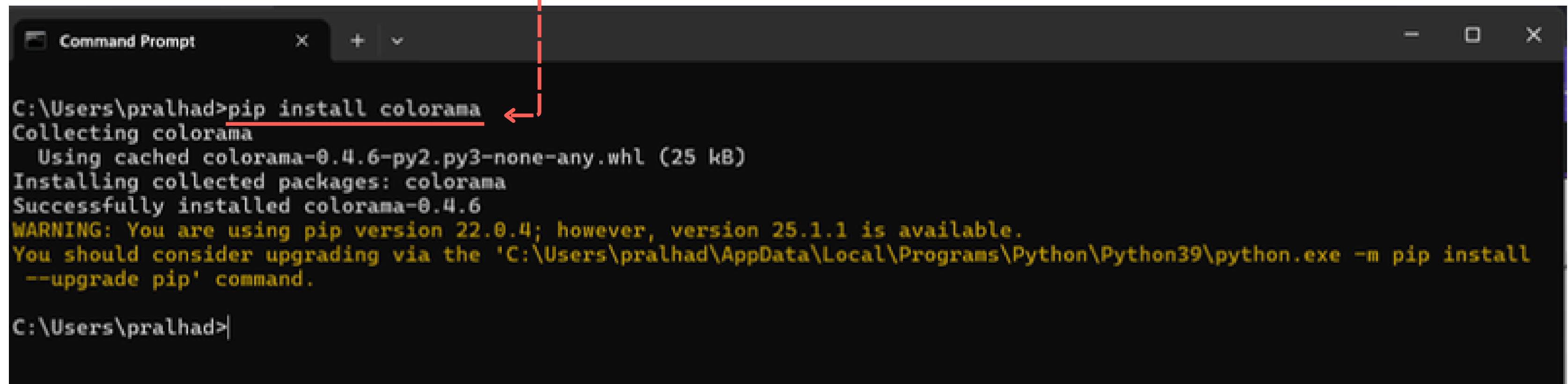
ON THIS PAGE

- Supported Methods
 - `ensurepip`
 - `get-pip.py`
 - Standalone zip application
- Alternative Methods
- Upgrading via
- Compatibility

stable

How to Install a Package Using pip

- Open your **terminal** (macOS/Linux) or **Command Prompt** (Windows)
- Type the command: 

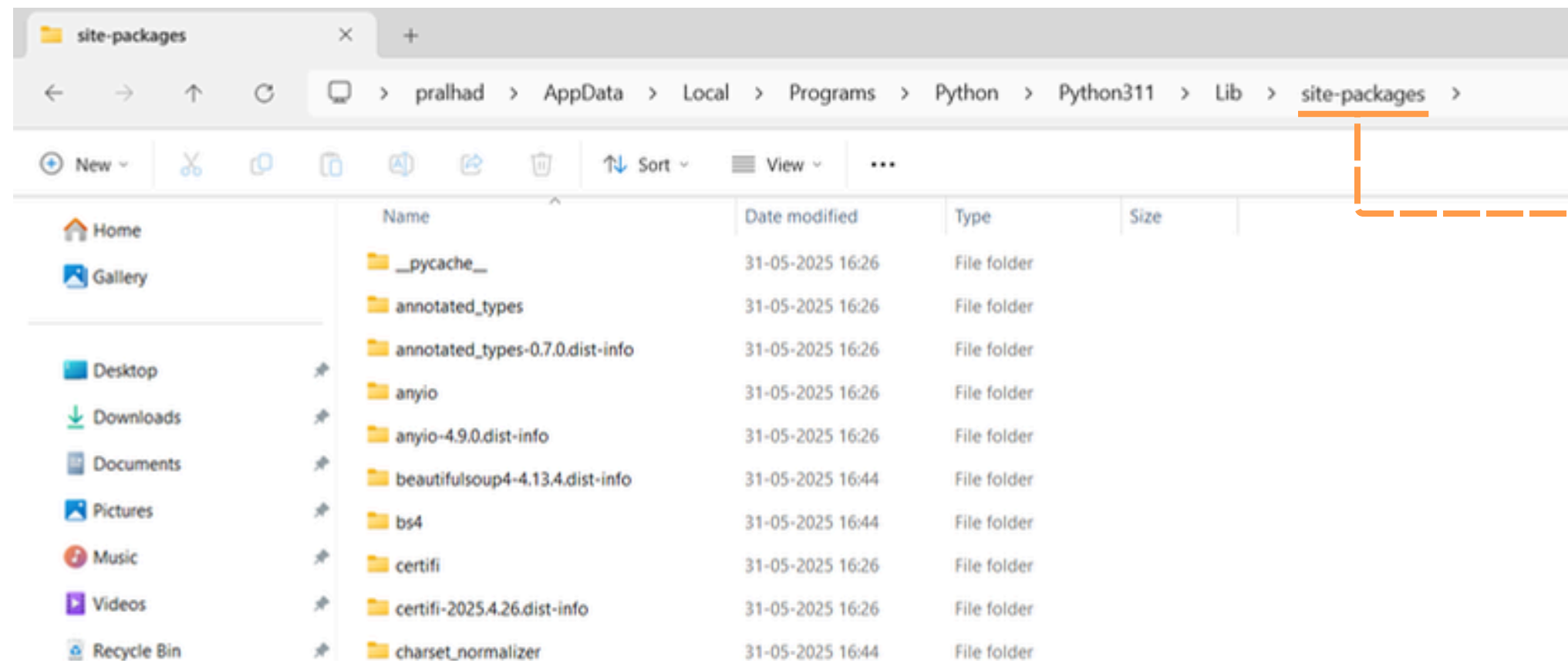


```
Command Prompt
C:\Users\pralhad>pip install colorama
Collecting colorama
  Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: colorama
Successfully installed colorama-0.4.6
WARNING: You are using pip version 22.0.4; however, version 25.1.1 is available.
You should consider upgrading via the 'C:\Users\pralhad\AppData\Local\Programs\Python\Python39\python.exe -m pip install --upgrade pip' command.
C:\Users\pralhad>
```

Where does pip Install Packages?

System Environment

- This installs packages **globally** on **your machine**.
- The **library** will be **available** to **all Python scripts** on **your system**.



in this folder pip install all packages

Windows Path → C:\Users\pralhad\AppData\Roaming\Python\Python310\site-packages



Install colorama



Objective

- Learn how to search for, verify, and install a Python library (**colorama**) using pip from the command line.



Instructions:

To complete this task:

- Search for the **colorama** library online to verify it's the correct package.
- Use the **terminal or command prompt** to install it using **pip**.
- **Verify installation** after it completes.

Using colorama in Python

- Changing Text Color
- Adding Color to Terminal Text
- Uninstalling Third-Party Libraries
- Key Differences Between Default Libraries and Third-Party Libraries

Changing Text Color

let's modify the example slightly. This time, we'll print "Hello World" in **red** text, and then **reset the style**.

text color

```
from colorama import Fore, Style
```

text Style options like dim, light, reset

```
print(Fore.RED + "Hello World")
```

```
print(Style.RESET_ALL)
```

It reset text style

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\pralhad\Desktop\merge_confict_demo> & C:/Users/pralhad/AppData/Local/Programs/Microsoft VS Code/bin/python t_demo/t.py  
Hello World
```

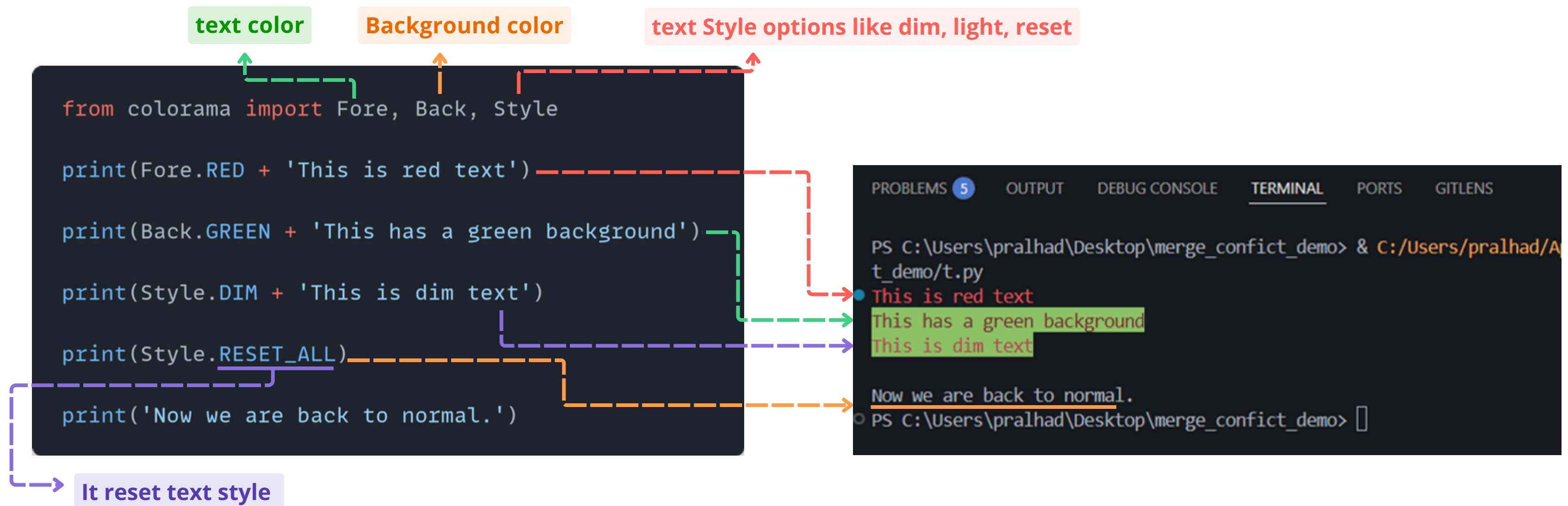
```
PS C:\Users\pralhad\Desktop\merge_confict_demo> 
```

Why Resetting Is Important

- Without **Style.RESET_ALL**, the style might carry over into any other text printed afterward.
- That's why it's a good practice to always reset styles once you're done.

Adding Color to Terminal Text

- colorama is a third-party Python library that lets you style text in the terminal. You can change:
- Text color
- Background color
- Text style → like dim or bright



Uninstalling Third-Party Libraries

You might want to **remove** a third-party package if:

- You no longer use it
- You're switching to a different library
- You need to clean up your environment

Python makes this easy using the **same tool** you used to install it — **pip**.

```
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>pip uninstall colorama
Found existing installation: colorama 0.4.6
Uninstalling colorama-0.4.6:
  Would remove:
    c:\users\pralhad\appdata\local\programs\python\python39\lib\site-packages\colorama-0.4.6.dist-info\*
    c:\users\pralhad\appdata\local\programs\python\python39\lib\site-packages\colorama\*
Proceed (Y/n)? Y
Successfully uninstalled colorama-0.4.6

C:\Users\pralhad>
```

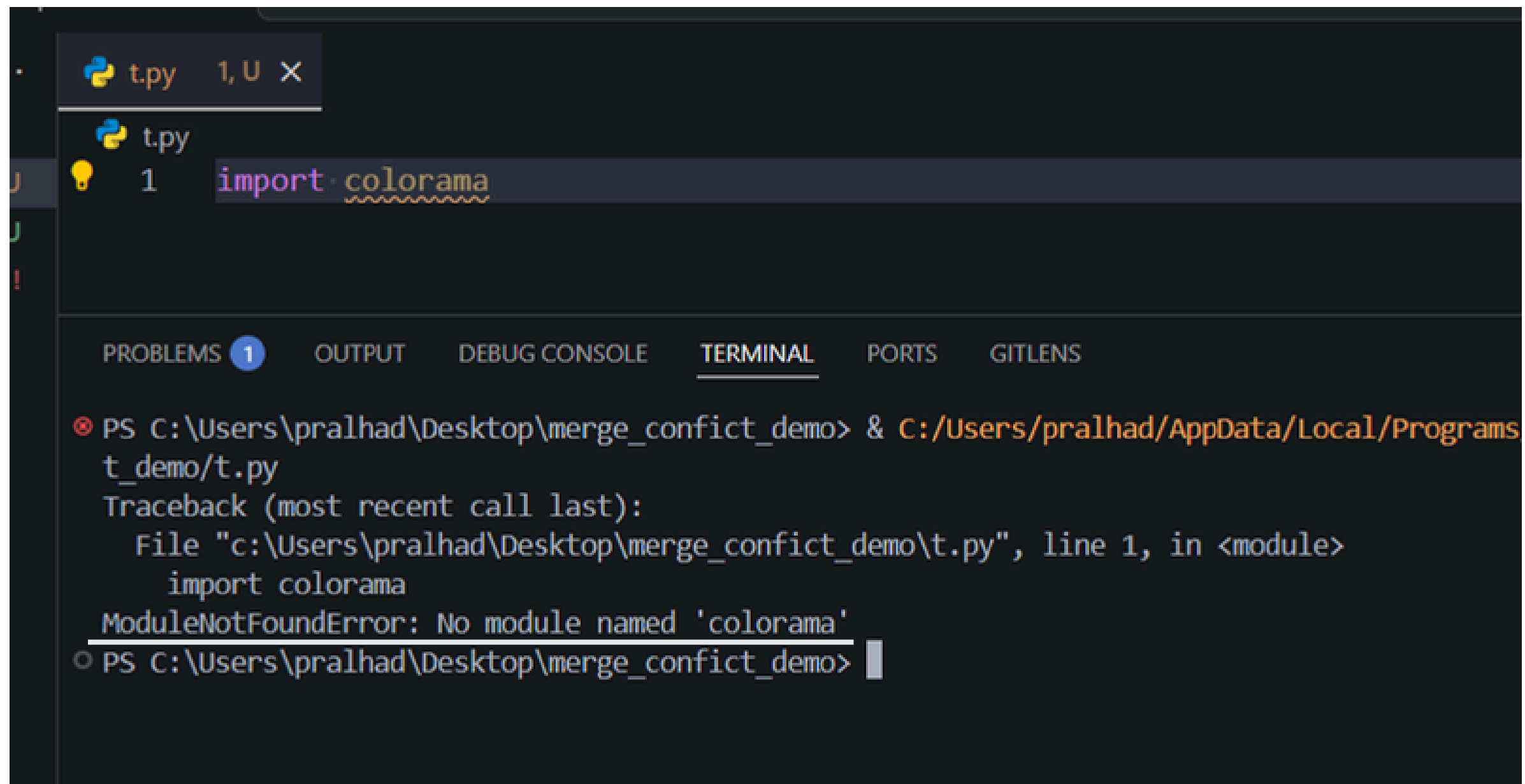
use to unstage any Libraries

Type y and press Enter to confirm.

✓ This will completely **remove** colorama from your environment.

Verifying Uninstallation

To make sure **colorama** has been **uninstalled properly**, let's try importing it in a new **Python script**.



The screenshot shows a VS Code editor with a file named `t.py` containing the following code:

```
1 import colorama
```

Below the editor, the **TERMINAL** tab is active, showing the command prompt output:

```
PS C:\Users\pralhad\Desktop\merge_confict_demo> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39-64/python.exe t_demo/t.py
Traceback (most recent call last):
  File "c:\Users\pralhad\Desktop\merge_confict_demo\t.py", line 1, in <module>
    import colorama
ModuleNotFoundError: No module named 'colorama'
PS C:\Users\pralhad\Desktop\merge_confict_demo>
```

This means the **library** has been **removed**, and you can no longer use it in your **Python scripts**.

Key Differences Between Default Libraries and Third-Party Libraries

Let's explore the main differences between **default libraries** (also called standard libraries) and **third-party libraries** in Python.

Feature	Default Libraries	Third-Party Libraries
• Installation	→ Pre-installed with Python	→ Must be installed separately using pip
• Source	→ Developed and maintained by the Python core team	→ Created by community developers or external teams
• Use Cases	→ Provides basic functionalities like file handling, mathematical operations, and more	→ Specialized tasks: data science, web, VFX, etc.
• Updates	→ Automatically updated with Python releases	→ Manually updated using pip install --upgrade
• Availability	→ Immediately available after Python install	→ Available from PyPI
• Examples	→ os, sys, math, datetime	→ colorama, pandas, requests, flask

Managing Packages with pip

- Basic Usage of pip
- Various pip Operations
- Popular VFX Third-Party Libraries
- Troubleshooting pip Installations

Basic Usage of pip

When working on **Python projects** — especially in VFX, **automation**, or **data tasks** — you'll often need to:

- Install new libraries
- Keep them up to date
- Match specific versions for compatibility

Installing a package:

- **Syntax** → `pip install package_name`
- **Example** → `pip install requests`

Installing a specific version of a package:

- **Syntax** → `pip install package_name==version`
- **Example** → `pip install numpy==1.21.0`

Upgrading a package:

- **Syntax** → `pip install --upgrade package_name`
- **Example** → `pip install --upgrade pandas`

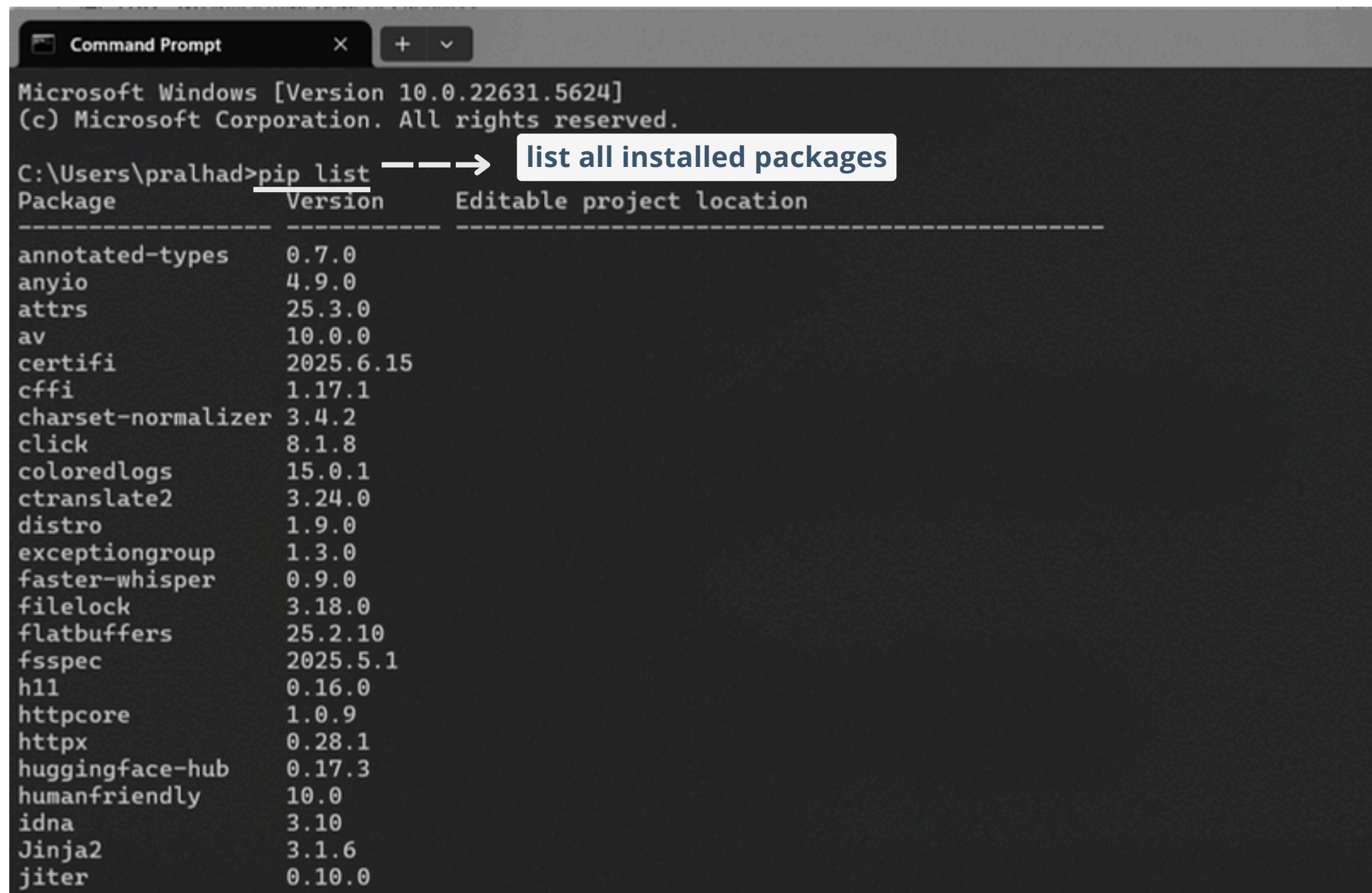
Uninstalling a package:

- **Syntax** → `pip uninstall package_name`
- **Example** → `pip uninstall colorama`

Various pip Operations

List Installed Packages

- Use this to see all packages installed in your environment, along with their **versions**.



```
Microsoft Windows [Version 10.0.22631.5624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>pip list
```

Package	Version	Editable project location
annotated-types	0.7.0	
anyio	4.9.0	
attrs	25.3.0	
av	10.0.0	
certifi	2025.6.15	
cffi	1.17.1	
charset-normalizer	3.4.2	
click	8.1.8	
coloredlogs	15.0.1	
ctranslate2	3.24.0	
distro	1.9.0	
exceptiongroup	1.3.0	
faster-whisper	0.9.0	
filelock	3.18.0	
flatbuffers	25.2.10	
fsspec	2025.5.1	
h11	0.16.0	
httpcore	1.0.9	
httpx	0.28.1	
huggingface-hub	0.17.3	
humanfriendly	10.0	
idna	3.10	
Jinja2	3.1.6	
jiter	0.10.0	

Checking Package Detailed Information

- Use this to view detailed information about a single installed package.

```
Command Prompt
C:\Users\pralhad>pip show requests
Name: requests
Version: 2.31.0
Summary: Python HTTP for Humans.
Home-page: https://requests.readthedocs.io
Author: Kenneth Reitz
Author-email: me@kennethreitz.org
License: Apache 2.0
Location: c:\users\pralhad\appdata\local\programs\python\python39\lib\site-packages
Requires: certifi, charset-normalizer, idna, urllib3
Required-by: huggingface-hub, tiktoken

C:\Users\pralhad>
```


Check for Outdated Packages

- This **shows** which **installed packages** have updates available.

```
Command Prompt
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>pip list --outdated --→ flag to view outdated packages
Package            Version    Latest    Type
-----
av                  10.0.0     15.0.0    wheel
certifi             2025.6.15  2025.7.14 wheel
ctranslate2         3.24.0     4.6.0     wheel
faster-whisper      0.9.0      1.1.1     wheel
fsspec              2025.5.1   2025.7.0  wheel
huggingface-hub     0.17.3     0.33.4    wheel
mypy                1.16.1     1.17.0    wheel
numpy               1.24.3     2.0.2     wheel
openai              1.93.0     1.97.0    wheel
openai-whisper      20231117   20250625  sdist
pip                 22.0.4     25.1.1    wheel
PyAudio             0.2.13     0.2.14    wheel
pydantic_core       2.33.2     2.35.2    wheel
python-dotenv       1.0.0      1.1.1     wheel
pywin32             310        311       wheel
requests            2.31.0     2.32.4    wheel
rich                 13.4.2     14.0.0    wheel
selenium            4.34.0     4.34.2    wheel
setuptools          58.1.0     80.9.0    wheel
sounddevice         0.4.6      0.5.2     wheel
tokenizers          0.14.1     0.21.2    wheel
torch               2.0.1      2.7.1     wheel
urllib3             2.4.0      2.5.0     wheel

WARNING: You are using pip version 22.0.4; however, version 25.1.1 is available.
You should consider upgrading via the 'C:\Users\pralhad\AppData\Local\Programs\Python\Py
```



Checking Installed Libraries with pip list



Objective

- Learn how to view all Python libraries installed in your environment using **pip list** and confirm successful installations like **colorama**.



Instructions:

To complete this task:

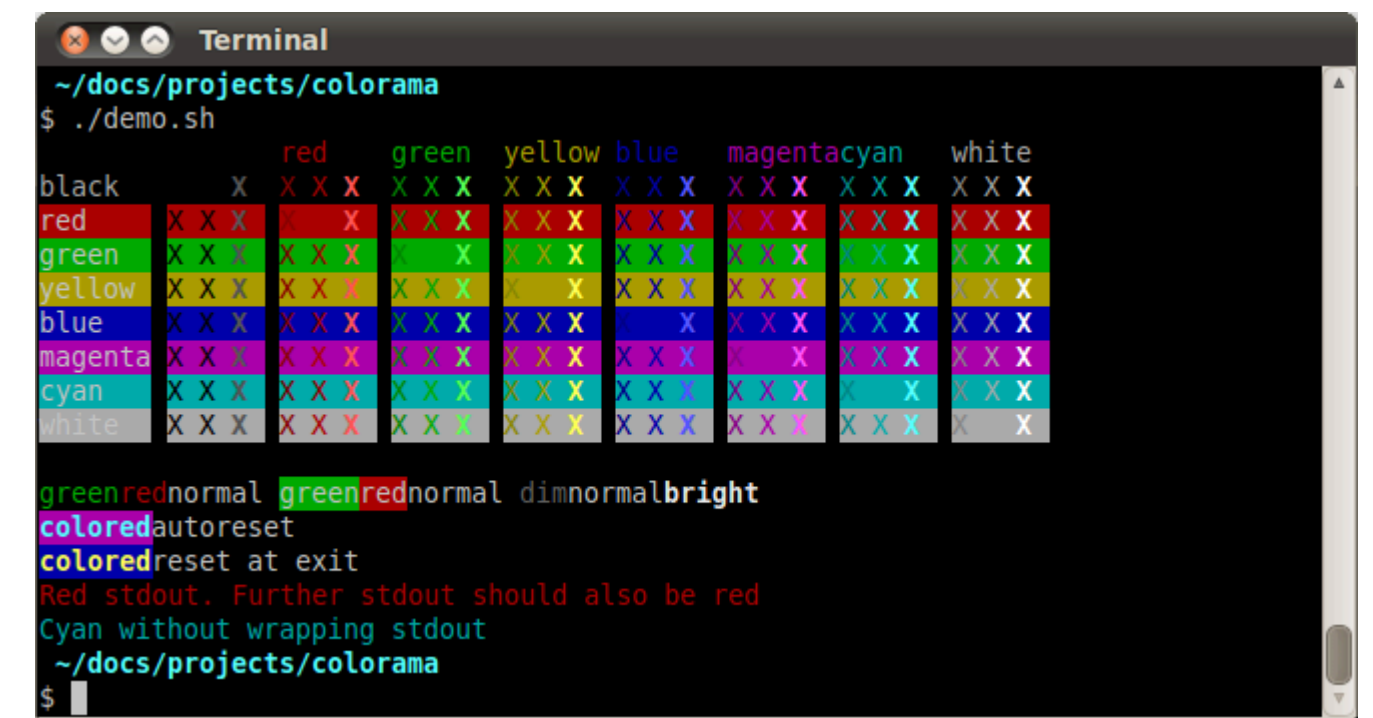
- Open your terminal or command prompt.
- Run the **pip list** command.
- View the list of installed libraries and check if **colorama** appears.
- Use this technique to confirm installations and library versions.

Popular VFX Third-Party Libraries

Here are some popular **third-party Python libraries** commonly used in **VFX pipelines**, with one-line descriptions and their applications in the pipeline:

Colorama

- **Description:** Adds color and style to terminal text.
- **Uses in VFX Pipeline:**
 - Enhances CLI tools with color-coded outputs
 - e.g., green for success, red for errors
 - Makes debugging and logging easier by improving visual clarity.
 - Useful in scripts that run batch jobs or automated checks.



```
Terminal
~/docs/projects/colorama
$ ./demo.sh

black      x  x x x  x x x  x x x  x x x  x x x  x x x  x x x
red        x x x  x x x  x x x  x x x  x x x  x x x  x x x
green      x x x  x x x  x x x  x x x  x x x  x x x  x x x
yellow     x x x  x x x  x x x  x x x  x x x  x x x  x x x
blue       x x x  x x x  x x x  x x x  x x x  x x x  x x x
magenta    x x x  x x x  x x x  x x x  x x x  x x x  x x x
cyan       x x x  x x x  x x x  x x x  x x x  x x x  x x x
white      x x x  x x x  x x x  x x x  x x x  x x x  x x x

greenrednormal greenrednormal dimnormalbright
colored autoreset
colored reset at exit
Red stdout. Further stdout should also be red
Cyan without wrapping stdout
~/docs/projects/colorama
$
```


Pillow (PIL Fork)

- **Description:** Image processing library for manipulating and analyzing images.
- **Uses in VFX Pipeline:**
 - Automates **resizing, format conversion**, and **validation** for **image assets**.
 - Enables quick previews or validation of frame sequences.
 - Lightweight and useful in **pre-render** or **review tools**.

PySide2 / PyQt

- **Description:** GUI libraries for building cross-platform desktop applications.
- **Uses in VFX Pipeline:**
 - Used to develop **custom artist-facing tools**, such as:
 - **Asset browsers**
 - **Submission forms**
 - **Render control panels**
 - Allows teams to build **interactive, standalone applications** for tool deployment.
 - Fully integrates with **Qt Designer** for **faster UI development**.

PyYAML

- **Description:** Library for parsing and writing **YAML** files.
- **Uses in VFX Pipeline:**
 - Stores **configuration data** like project settings, paths, or tool preferences in readable format.
 - Makes it easy to **customize tools per project or department**.
 - Helps in **version-controlling environment configs** in studio pipelines.

Requests

- **Description:** Simplifies **sending HTTP requests** to **web services** or **APIs**.
- **Uses in VFX Pipeline:**
 - Communicates with **production trackers, render farms, or asset servers** via **REST APIs**.
 - Sends and receives metadata from **cloud services** or **pipeline web dashboards**.
 - Often used to **automate asset publishing, status checks, or notifications**.

Troubleshooting pip Installations

- Here's a detailed explanation of **common errors** encountered when **using pip** and how to troubleshoot them:

Permission Denied

● Issue

- You get a "**Permission Denied**" error when **installing** a package globally.

💡 Cause:

- You don't have **admin rights** to install **system-wide packages**.

Command:

```
pip install --user requests
```

flag → use to install package user-specific directory:

Explanation:

- This installs the package in a user-specific directory:
 - **Linux/macOS:** ~/.local/lib/pythonX.Y/site-packages
 - **Windows:** C:\Users\<username>\AppData\Roaming\Python\...
- No admin access is needed.

Proxy Issues

● Issue

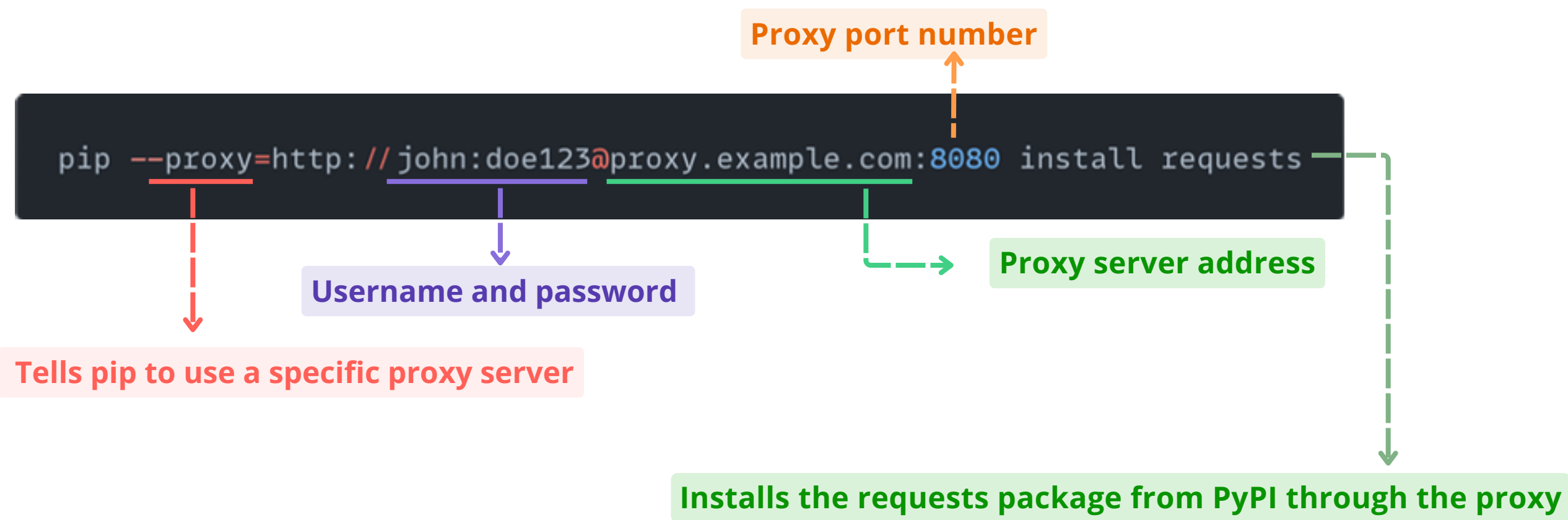
- You get timeout or connection errors when pip tries to connect to the **Python Package Index (PyPI)**.

💡 Cause:

- Your internet connection is routed through a **proxy server**.

Command:

```
pip --proxy=http://john:doe123@proxy.example.com:8080 install requests
```



Proxy port number

Username and password

Proxy server address

Tells pip to use a specific proxy server

Installs the requests package from PyPI through the proxy

Concept Practice Task

- Python Libraries

Create nested directories

```
import os

# Create a nested directory structure
nested_path = "projects/python/scripts"

os.makedirs(nested_path, exist_ok=True)
print(f"Nested directories created: {nested_path}")
```

List all files in a directory

```
import os

# List all files in the current directory
files = os.listdir(".")

print("Files and directories in the current folder:")
for file in files:
    print(file)
```

Get the size of a directory

```
import os

def get_directory_size(directory):
    total_size = 0

    for dirpath, dirnames, filenames in os.walk(directory):
        for file in filenames:
            file_path = os.path.join(dirpath, file)
            total_size += os.path.getsize(file_path)
    return total_size

# Get the size of the current directory
directory_size = get_directory_size(".")
print(f"Total size of the current directory: {directory_size / 1024:.2f} KB")
```

Add a timestamp to a file name

```
import datetime

# Generate a timestamp
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

# File name with timestamp
file_name = f"report_{timestamp}.txt"
print(f"File name with timestamp: {file_name}")
```

Identify the operating system

```
import os

# Check the name of the operating system
os_name = os.name

if os_name == "nt":
    print("You are using Windows.")
elif os_name == "posix":
    print("You are using a Unix-based system.")
else:
    print("Unknown operating system.")
```

Rename a file

```
import os

# Create a sample file
file_name = "example.txt"

with open(file_name, "w") as file:
    file.write("Sample content.")

# Rename the file
new_name = "renamed_example.txt"
os.rename(file_name, new_name)

print(f"File renamed to: {new_name}")
```

Thank You

